

grok-wsl-ubuntu / 019f45ba-a4b9-7290-bd20-fa42b34d67a9

Metadata

```
- Source: `grok-wsl-ubuntu`  
- Kind: `grok`  
- Source file:  
`\\wsl.localhost\Ubuntu\home\avidullu\.grok\sessions\%2Fhome%2Favidullu%2Fprojects%2Fkhelsutra-guru\019f45ba-a4b9-7290-bd20-fa42b34d67a9\chat_history.jsonl`  
- SHA-256: `b0cebdffc6ad02b3e6c0249a6081c687c3aaa5b972613b8fb0557d468fbcc1a42`  
- Source modified: `2026-07-09T07:47:47+00:00`  
- Imported at: `2026-07-09T08:32:32+00:00`  
- project: `%2Fhome%2Favidullu%2Fprojects%2Fkhelsutra-guru`  
- session_id: `019f45ba-a4b9-7290-bd20-fa42b34d67a9`
```

Transcript

1. system

You are Grok 4.5 released by xAI. You are an interactive CLI tool that helps users with software engineering tasks. Your main goal is to complete the user's request, denoted within the `<user_query>` tag.

`<executing_actions_with_care>`

Carefully consider the reversibility and blast radius of actions. Generally you can freely take local, reversible actions like editing files or running tests. But for actions that are hard to reverse, affect shared systems beyond your local environment, or could otherwise be risky or destructive, check with the user before proceeding. The cost of pausing to confirm is low, while the cost of an unwanted action (lost work, unintended messages sent, deleted branches) can be very high. For actions like these, consider the context, the action, and user instructions, and by default transparently communicate the action and ask for confirmation before proceeding. This default can be changed by user instructions - if explicitly asked to operate more autonomously, then you may proceed without confirmation, but still attend to the risks and consequences when taking actions. A user approving an action (like a git push) once does NOT mean that they approve it in all contexts, so unless actions are authorized in advance in durable instructions like AGENTS.md files, always confirm first. Authorization stands for the scope specified, not beyond. Match the scope of your actions to what was actually requested.

Examples of the kind of risky actions that warrant user confirmation:

- Destructive operations: deleting files/branches, dropping database tables, killing processes, `rm -rf`, overwriting uncommitted changes
- Hard-to-reverse operations: force-pushing (can also overwrite upstream), `git reset --hard`, amending published commits, removing or downgrading packages/dependencies, modifying CI/CD pipelines
- Actions visible to others or that affect shared state: pushing code, creating/closing/commenting on PRs or issues, sending messages (Slack, email, GitHub), posting to external services, modifying shared infrastructure or permissions
- Uploading content to third-party web tools (diagram renderers, pastebins, gists) publishes it
- consider whether it could be sensitive before sending, since it may be cached or indexed even if later deleted.

When you encounter an obstacle, do not use destructive actions as a shortcut to simply make it go away. For instance, try to identify root causes and fix underlying issues rather than bypassing safety checks (e.g. `--no-verify`). If you discover unexpected state like unfamiliar files, branches, or configuration, investigate before deleting or overwriting, as it may represent the user's in-progress work. For example, typically resolve merge conflicts rather than discarding changes; similarly, if a lock file exists, investigate what process holds it rather than deleting it. In short: only take risky actions carefully, and when in doubt, ask before acting. Follow both the spirit and letter of these instructions - measure twice, cut once.

`</executing_actions_with_care>`

`<tool_calling>`

- Use specialized tools instead of bash commands when possible, as this provides a better user experience. For file operations, prefer dedicated file tools (e.g., `read_file` for reading files instead of `cat/head/tail`, `search_replace` for editing and creating files instead of `sed/awk`). Reserve bash tools exclusively for actual system commands and terminal operations that require shell execution. NEVER use bash `echo` or other command-line tools to communicate thoughts, explanations, or instructions to the user. Output all communication directly in your response text instead.

</tool_calling>

<background_tasks>

For watch processes, polling, and ongoing observation (CI status, log tailing, API polling): Use the `monitor` tool ? it streams each stdout line back as a chat notification.

</background_tasks>

<output_efficiency>

- Write like an excellent technical blog post ? precise, well-structured, and clear, in complete sentences. Most responses should be concise and to the point, but the quality of prose should be high.

- Same standards for commit and PR descriptions: complete sentences, good grammar, and only relevant detail.

- Prefer simple, accessible language over dense technical jargon. Explain what changed and why in plain language rather than listing identifiers. Stay focused: avoid filler, repetition, over-the-top detail, and tangents the user did not ask for.

- Keep final responses proportional to task complexity.

</output_efficiency>

<formatting>

Your text output is rendered as GitHub-flavored markdown (CommonMark). Use markdown actively when it aids the reader: bullet lists for parallel items, **bold** for emphasis, `inline code` for identifiers/paths/commands, and tables for short enumerable facts (file/line/status, before/after, quantitative data). Don't pack explanatory reasoning into table cells ? explain before or after the table. Match structure to the task: a simple question gets a direct answer in prose, not headers and numbered sections.

</formatting>

<user_guide>

Documentation about the Grok Build TUI ? including configuration, keyboard shortcuts, MCP servers, skills, theming, plugins, and more ? is stored as `.md` files in `~/grok/docs/user-guide/`. When users ask about features or how to use the TUI, read the relevant file from that directory. Present the information directly.

</user_guide>

2. user

<user_info>

OS Version: linux
Shell: /bin/bash
Workspace Path: /home/avidullu/projects/khelsutra-guru
Today's date: 2026-07-09
Note: Prefer using relative paths over absolute paths as tool call args when possible.

</user_info>

3. user

<system-reminder>

The following skills are available for use:

- `check-work`: Check your work with a verification subagent that reviews diffs, runs builds and tests, and evaluates correctness. Read this file for instructions
Use when: asked to "check work", "verify changes", "self-verify", "/check-work", "/check", "/verify", or "/self-verify".
Absolute path: /home/avidullu/.grok/skills/check-work/SKILL.md

- `pptx`: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like

in an email or summary); editing, modifying, or updating existing presentations; combining or splitting slide files; work?

Absolute path: /home/avidullu/.grok/skills/pptx/SKILL.md

- create-skill: Interactively create a new Grok skill (SKILL.md + optional scripts/references)
Use when: the user wants to create a skill, scaffold a skill, or runs /create-skill.

Absolute path: /home/avidullu/.grok/skills/create-skill/SKILL.md

- xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger espec?

Absolute path: /home/avidullu/.grok/skills/xlsx/SKILL.md

- help: Grok documentation and configuration help

Use when: users ask about setup, configuration, MCP servers, authentication, skills, slash commands, keyboard shortcuts, or any Grok feature. Also use proactively when you detect a user is having trouble with setup or onboarding.

Absolute path: /home/avidullu/.grok/skills/help/SKILL.md

- image: How to use the image_gen and image_edit tool calls in Grok Build: when to build a visual with code instead of generating it, prompt-craft, reference-first handling of real people, factual grounding, and asset-consistency. Load this whenever generating or editing an image is on the table, i.e. when an image_gen or image_edit call is being considered or about to be made. Tool-usage-driven, not tr?

Absolute path: /home/avidullu/.grok/skills/image/SKILL.md

- docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also use when extracting or reorganizing content from .docx files, inserting or replacing ima?

Absolute path: /home/avidullu/.grok/skills/docx/SKILL.md

- review: Run a reviewer subagent against uncommitted local changes, a named branch, or a GitHub PR. Local and branch modes write a review file plus a summary to disk. PR mode posts the findings as a PENDING GitHub review for the user to inspect and submit through the UI.

Use when: asked to 'review', 'code review', 'review my changes', 'review this PR', or '/review'.

Absolute path: /home/avidullu/.grok/bundled/skills/review/SKILL.md

- pr-babysit: Monitor PRs, fix CI failures, address review comments, resolve merge conflicts, and restack stacks. Supports independent PRs, Graphite stacks, and GitHub stacked PRs (gh-stack).

Use when: "/pr-babysit".

Absolute path: /home/avidullu/.grok/bundled/skills/pr-babysit/SKILL.md

- design: Run the full design-doc-writer and design-doc-reviewer loop until consensus. Produces a polished design document with a PR plan.

Use when: asked to "design", "write a design doc", "system design", "architecture doc", "technical spec", or "/design".

Absolute path: /home/avidullu/.grok/bundled/skills/design/SKILL.md

- execute-plan: Execute a PR Plan DAG from a design document. Parses the plan, topologically sorts it, implements PRs in parallel using worktree-isolated subagents, runs mandatory orchestrator-level review, and assembles either a Graphite PR stack or a plain-git branch stack depending on tool availability.

Use when: asked to "execute plan", "run the plan", "implement the design", or "/execute-plan".

Absolute path: /home/avidullu/.grok/bundled/skills/execute-plan/SKILL.md

</system-reminder>

4. user

<system-reminder>

MCP servers connected:

- github (91 tools)
- gmail (16 tools)
- google_calendar (8 tools)
- google_drive (5 tools)
- outlook (18 tools)
- tasks (6 tools)

To use MCP tools, you MUST call `search\_tool` first to retrieve the tool's input schema before

calling `use\_tool`. NEVER guess parameter names ? always use the exact schema returned by `search\_tool`.

</system-reminder>

5. user

<user_query>
hello. Can you ramp up on the <https://github.com/Khelsutra/badminton-highlight-indexer/pull/550> and review it. All the code is within this root folder as repos inside each directory for your reference
</user_query>

6. assistant

I'll review PR #550. Loading the review skill and locating the local repos that map to it.

7. tool_result

```
{
  "results": [
    {
      "server": "github",
      "tools": [
        {
          "tool_name": "github__request_copilot_review",
          "description": "Request a GitHub Copilot code review for a pull request. Use this for automated feedback on pull requests, usually before requesting a human reviewer.",
          "score": 12.814598083496094,
          "input_schema": {
            "type": "object",
            "properties": {
              "owner": {
                "type": "string",
                "description": "Repository owner",
                "x-mcp-header": "owner"
              },
              "pullNumber": {
                "type": "number",
                "description": "Pull request number"
              },
              "repo": {
                "type": "string",
                "description": "Repository name",
                "x-mcp-header": "repo"
              }
            }
          },
          "required": [
            "owner",
            "repo",
            "pullNumber"
          ]
        }
      ]
    },
    {
      "tool_name": "github__pull_request_review_write",
      "description": "Create and/or submit, delete review of a pull request.\n\nAvailable methods:\n- create: Create a new review of a pull request. If \"event\" parameter is provided, the review is submitted. If \"event\" is omitted, a pending review is created.\n- submit_pending: Submit an existing pending review of a pull request. This requires that a pending review exists for the current user on the specified pull request. The \"body\" and \"event\" parameters are used when submitting the review.\n- delete_pending: Delete an existing pending review of a pull request. This requires that a pending review exists for the current user on the specified pull request.\n- resolve_thread: Resolve a review thread. Requires only \"threadId\" parameter with the thread's node ID (e.g., PRRT_kwD0xxx). The owner, repo, and
```

pullNumber parameters are not used for this method. Resolving an already-resolved thread is a no-op.\n- unresolve_thread: Unresolve a previously resolved review thread. Requires only \n\"threadId\" parameter. The owner, repo, and pullNumber parameters are not used for this method. Unresolving an already-unresolved thread is a no-op.\n",

```
"score": 11.834545135498047,
"input_schema": {
  "type": "object",
  "properties": {
    "body": {
      "type": "string",
      "description": "Review comment text"
    },
    "commitID": {
      "type": "string",
      "description": "SHA of commit to review"
    },
    "event": {
      "type": "string",
      "description": "Review action to perform.",
      "enum": [
        "APPROVE",
        "REQUEST_CHANGES",
        "COMMENT"
      ]
    },
    "method": {
      "type": "string",
      "description": "The write operation to perform on pull request review.",
      "enum": [
        "create",
        "submit_pending",
        "delete_pending",
        "resolve_thread",
        "unresolve_thread"
      ]
    },
    "owner": {
      "type": "string",
      "description": "Repository owner",
      "x-mcp-header": "owner"
    },
    "pullNumber": {
      "type": "number",
      "description": "Pull request number"
    },
    "repo": {
      "type": "string",
      "description": "Repository name",
      "x-mcp-header": "repo"
    },
    "threadId": {
      "type": "string",
      "description": "The node ID of the review thread (e.g., PRRT_kwDOxxx). Required for resolve_thread and unresolve_thread methods. Get thread IDs from pull_request_read with method get_review_comments."
    }
  },
  "required": [
    "method",
    "owner",
    "repo",
    "pullNumber"
  ]
}
```

```

{
  "tool_name": "github__update_pull_request",
  "description": "Update an existing pull request in a GitHub repository.",
  "score": 11.342342376708984,
  "input_schema": {
    "type": "object",
    "properties": {
      "base": {
        "type": "string",
        "description": "New base branch name"
      },
      "body": {
        "type": "string",
        "description": "New description"
      },
      "draft": {
        "type": "boolean",
        "description": "Mark pull request as draft (true) or ready for review (false)"
      },
      "maintainer_can_modify": {
        "type": "boolean",
        "description": "Allow maintainer edits"
      },
      "owner": {
        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
      },
      "pullNumber": {
        "type": "number",
        "description": "Pull request number to update"
      },
      "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
      },
      "reviewers": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "GitHub usernames or ORG/team-slug team reviewers to request
reviews from"
      },
      "state": {
        "type": "string",
        "description": "New state",
        "enum": [
          "open",
          "closed"
        ]
      },
      "title": {
        "type": "string",
        "description": "New title"
      }
    },
    "required": [
      "owner",
      "repo",
      "pullNumber"
    ]
  }
}

```

```

{
  "tool_name": "github__create_pull_request",
  "description": "Create a new pull request in a GitHub repository.",
  "score": 11.15595817565918,
  "input_schema": {
    "type": "object",
    "properties": {
      "base": {
        "type": "string",
        "description": "Branch to merge into"
      },
      "body": {
        "type": "string",
        "description": "PR description"
      },
      "draft": {
        "type": "boolean",
        "description": "Create as draft PR"
      },
      "head": {
        "type": "string",
        "description": "Branch containing changes"
      },
      "maintainer_can_modify": {
        "type": "boolean",
        "description": "Allow maintainer edits"
      },
      "owner": {
        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
      },
      "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
      },
      "reviewers": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "GitHub usernames or ORG/team-slug team reviewers to request
reviews from"
      },
      "title": {
        "type": "string",
        "description": "PR title"
      }
    },
    "required": [
      "owner",
      "repo",
      "title",
      "head",
      "base"
    ]
  }
},
{
  "tool_name": "github__add_comment_to_pending_review",
  "description": "Add review comment to the requester's latest pending pull request
review. A pending review needs to already exist to call this (check with the user if not
sure).",
  "score": 10.753103256225586,

```

```

"input_schema": {
  "type": "object",
  "properties": {
    "body": {
      "type": "string",
      "description": "The text of the review comment"
    },
    "line": {
      "type": "number",
      "description": "The line of the blob in the pull request diff that the comment
applies to. For multi-line comments, the last line of the range"
    },
    "owner": {
      "type": "string",
      "description": "Repository owner",
      "x-mcp-header": "owner"
    },
    "path": {
      "type": "string",
      "description": "The relative path to the file that necessitates a comment"
    },
    "pullNumber": {
      "type": "number",
      "description": "Pull request number"
    },
    "repo": {
      "type": "string",
      "description": "Repository name",
      "x-mcp-header": "repo"
    },
    "side": {
      "type": "string",
      "description": "The side of the diff to comment on. LEFT indicates the previous
state, RIGHT indicates the new state",
      "enum": [
        "LEFT",
        "RIGHT"
      ]
    },
    "startLine": {
      "type": "number",
      "description": "For multi-line comments, the first line of the range that the
comment applies to"
    },
    "startSide": {
      "type": "string",
      "description": "For multi-line comments, the starting side of the diff that the
comment applies to. LEFT indicates the previous state, RIGHT indicates the new state",
      "enum": [
        "LEFT",
        "RIGHT"
      ]
    },
    "subjectType": {
      "type": "string",
      "description": "The level at which the comment is targeted",
      "enum": [
        "FILE",
        "LINE"
      ]
    }
  },
  "required": [
    "owner",
    "repo",

```



```

        "pullNumber",
        "path",
        "body",
        "subjectType"
    ]
}
},
{
    "tool_name": "github__add_issue_comment",
    "description": "Add a comment and/or reaction to a specific issue or issue comment in
a GitHub repository. Use this tool with pull requests as well (in this case pass pull request
number as issue_number), but only if user is not asking specifically to add or react to review
comments. At least one of body or reaction is required.",
    "score": 8.759724617004395,
    "input_schema": {
        "type": "object",
        "properties": {
            "body": {
                "type": "string",
                "description": "Comment content. Required unless reaction is provided."
            },
            "comment_id": {
                "type": "number",
                "description": "The numeric ID of the issue or pull request comment to react to.
Use this for reactions to comments; omit it to react to the issue or pull request itself. Cannot
be combined with body.",
                "minimum": 1
            },
            "issue_number": {
                "type": "number",
                "description": "Issue or pull request number to comment on or react to."
            },
            "owner": {
                "type": "string",
                "description": "Repository owner",
                "x-mcp-header": "owner"
            },
            "reaction": {
                "type": "string",
                "description": "Emoji reaction to add. Required unless body is provided.",
                "enum": [
                    "+1",
                    "-1",
                    "laugh",
                    "confused",
                    "heart",
                    "hooray",
                    "rocket",
                    "eyes"
                ]
            },
            "repo": {
                "type": "string",
                "description": "Repository name",
                "x-mcp-header": "repo"
            }
        },
        "required": [
            "owner",
            "repo",
            "issue_number"
        ]
    }
},
{

```

```

        "tool_name": "github__list_notifications",
        "description": "Lists all GitHub notifications for the authenticated user, including
unread notifications, mentions, review requests, assignments, and updates on issues or pull
requests. Use this tool whenever the user asks what to work on next, requests a summary of their
GitHub activity, wants to see pending reviews, or needs to check for new updates or tasks. This
tool is the primary way to discover actionable items, reminders, and outstanding work on GitHub.
Always call this tool when asked what to work on next, what is pending, or what needs attention
in GitHub.",
        "score": 8.397501945495605,
        "input_schema": {
            "type": "object",
            "properties": {
                "before": {
                    "type": "string",
                    "description": "Only show notifications updated before the given time (ISO 8601
format)"
                },
                "filter": {
                    "type": "string",
                    "description": "Filter notifications to, use default unless specified. Read
notifications are ones that have already been acknowledged by the user. Participating
notifications are those that the user is directly involved in, such as issues or pull requests
they have commented on or created.",
                    "enum": [
                        "default",
                        "include_read_notifications",
                        "only_participating"
                    ]
                },
                "owner": {
                    "type": "string",
                    "description": "Optional repository owner. If provided with repo, only
notifications for this repository are listed.",
                    "x-mcp-header": "owner"
                },
                "page": {
                    "type": "number",
                    "description": "Page number for pagination (min 1)",
                    "minimum": 1
                },
                "perPage": {
                    "type": "number",
                    "description": "Results per page for pagination (min 1, max 100)",
                    "minimum": 1,
                    "maximum": 100
                },
                "repo": {
                    "type": "string",
                    "description": "Optional repository name. If provided with owner, only
notifications for this repository are listed.",
                    "x-mcp-header": "repo"
                },
                "since": {
                    "type": "string",
                    "description": "Only show notifications updated after the given time (ISO 8601
format)"
                }
            }
        },
        {
            "tool_name": "github__merge_pull_request",
            "description": "Merge a pull request in a GitHub repository.",
            "score": 7.606374740600586,
            "input_schema": {

```

```

    "type": "object",
    "properties": {
      "commit_message": {
        "type": "string",
        "description": "Extra detail for merge commit"
      },
      "commit_title": {
        "type": "string",
        "description": "Title for merge commit"
      },
      "merge_method": {
        "type": "string",
        "description": "Merge method",
        "enum": [
          "merge",
          "squash",
          "rebase"
        ]
      },
      "owner": {
        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
      },
      "pullNumber": {
        "type": "number",
        "description": "Pull request number"
      },
      "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
      }
    },
    "required": [
      "owner",
      "repo",
      "pullNumber"
    ]
  },
  {
    "tool_name": "github__pull_request_read",
    "description": "Get information on a specific pull request in GitHub repository.",
    "score": 7.606374740600586,
    "input_schema": {
      "type": "object",
      "properties": {
        "after": {
          "type": "string",
          "description": "Cursor for pagination, used only by the get_review_comments method. Pass the endCursor from the previous page's PageInfo to fetch the next page."
        },
        "method": {
          "type": "string",
          "description": "Action to specify what pull request data needs to be retrieved from GitHub. \nPossible options: \n 1. get - Get details of a specific pull request.\n 2. get_diff - Get the diff of a pull request.\n 3. get_status - Get combined commit status of a head commit in a pull request.\n 4. get_files - Get the list of files changed in a pull request. Use with pagination parameters to control the number of results returned.\n 5. get_commits - Get the list of commits on a pull request. Use with pagination parameters to control the number of results returned.\n 6. get_review_comments - Get review threads on a pull request. Each thread contains logically grouped review comments made on the same code location during pull request reviews. Returns threads with metadata (isResolved, isOutdated, isCollapsed) and their associated comments. Use cursor-based pagination (perPage, after) to control results.\n 7."
        }
      }
    }
  }
}

```

get_reviews - Get the reviews on a pull request. When asked for review comments, use get_review_comments method. Use with pagination parameters to control the number of results returned.\n 8. get_comments - Get comments on a pull request. Use this if user doesn't specifically want review comments. Use with pagination parameters to control the number of results returned.\n 9. get_check_runs - Get check runs for the head commit of a pull request. Check runs are the individual CI/CD jobs and checks that run on the PR.\n",

```

    "enum": [
      "get",
      "get_diff",
      "get_status",
      "get_files",
      "get_commits",
      "get_review_comments",
      "get_reviews",
      "get_comments",
      "get_check_runs"
    ],
  },
  "owner": {
    "type": "string",
    "description": "Repository owner",
    "x-mcp-header": "owner"
  },
  "page": {
    "type": "number",
    "description": "Page number for pagination (min 1)",
    "minimum": 1
  },
  "perPage": {
    "type": "number",
    "description": "Results per page for pagination (min 1, max 100)",
    "minimum": 1,
    "maximum": 100
  },
  "pullNumber": {
    "type": "number",
    "description": "Pull request number"
  },
  "repo": {
    "type": "string",
    "description": "Repository name",
    "x-mcp-header": "repo"
  }
},
"required": [
  "method",
  "owner",
  "repo",
  "pullNumber"
]
}
},
{
  "tool_name": "github_update_pull_request_branch",
  "description": "Update the branch of a pull request with the latest changes from the base branch.",
  "score": 7.574429512023926,
  "input_schema": {
    "type": "object",
    "properties": {
      "expectedHeadSha": {
        "type": "string",
        "description": "The expected SHA of the pull request's HEAD ref"
      },
      "owner": {

```

```

        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
    },
    "pullNumber": {
        "type": "number",
        "description": "Pull request number"
    },
    "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
    }
},
"required": [
    "owner",
    "repo",
    "pullNumber"
]
}
}
]
}
],
"total_hidden_tools": 144,
"status": "ready",
"note": null
}

```

8. tool_result

```

- /home/avidullu/projects/khelsutra-guru/
- Annotation Setup/
  - 6 June 2026/
    - badminton (1)/
      - annotations/
        - instances_Test.json
    - badminton (2)/
      - annotations.xml
    - badminton/
      [715 files in subtree: 710 *.xml, 5 *.txt]
  - Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf
- muneem/
  - SampleStatements/
    - HDFC_Avi_Dullu_31072024_150007641.pdf
  - Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf
- badminton-highlight-indexer/
  - __pycache__/
    - run_eval.cpython-312.pyc
    - run_regression.cpython-312.pyc
- backend/
  - __pycache__/
    [6 files in subtree: 6 *.pyc]
  - annotations/
    [8 files in subtree: 4 *.py, 4 *.pyc]
  - api/
    [13 files in subtree: 5 *.py, 5 *.pyc, 1 *.css, ...]
  - assets/
    - fonts/
      [6 files in subtree: 3 *.ttf, 2 *.txt, 1 *.md]
  - billing.py
  - compute/
    [6 files in subtree: 3 *.py, 3 *.pyc]
  - config/
    [12 files in subtree: 6 *.py, 6 *.pyc]

```

- eval/
 - [82 files in subtree: 41 *.py, 41 *.pyc]
- features/
 - [4 files in subtree: 2 *.py, 2 *.pyc]
- flywheel.py
- infrastructure/
 - [10 files in subtree: 5 *.py, 5 *.pyc]
- main.py
- orchestration.py
- pipeline/
 - [76 files in subtree: 38 *.py, 38 *.pyc]
- providers/
 - [6 files in subtree: 3 *.py, 3 *.pyc]
- reporting/
 - [7 files in subtree: 3 *.py, 3 *.pyc, 1 *.yaml]
- results.py
- server_runtime.py
- sports/
 - [6 files in subtree: 3 *.py, 3 *.pyc]
- storage/
 - [18 files in subtree: 9 *.py, 9 *.pyc]
- tools/
 - [6 files in subtree: 3 *.py, 3 *.pyc]
- ui/
 - [11 files in subtree: 6 *.woff2, 1 *.css, 1 *.html, ...]
- utils/
 - [28 files in subtree: 14 *.py, 14 *.pyc]
- worker/
 - [4 files in subtree: 2 *.py, 2 *.pyc]
- CLAUDE.md
- config.json
- deploy/
 - cloudrun/
 - [5 files in subtree: 1 *.md, 1 *.no-ext, 1 *.ps1, ...]
 - digitalocean/
 - [4 files in subtree: 3 *.sh, 1 *.md]
 - gcp/
 - [14 files in subtree: 9 *.sh, 2 *.md, 1 *.env, ...]
- docs/
 - ABLATION_LAYER.md
 - ALPHA_LAUNCH_READINESS.md
 - ALPHA_LAUNCH_REVIEW_2026-07-04.md
 - ALPHA_TESTER_ONBOARDING_FAQ.md
 - ANALYZERS_AND_RECONCILERS.md
 - archives/
 - [49 files in subtree: 48 *.md, 1 *.py]
 - BACKLOG.md
 - CLOUD_GPU_DRYRUN_GUIDE.md
 - CODE_MAP.md
 - COMMERCIALIZATION.md
 - COMPUTE_DECOUPLED_SERVING/
 - [16 files in subtree: 15 *.md, 1 *.svg]
 - CONTROL_PLANE_DURABLE_REBUILD.md
 - CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md
 - CREATING_REELS.md
 - DATA_IN_GCS.md
 - data_pipeline/
 - [6 files in subtree: 6 *.md]
 - DECISION_SNAPSHOT_REGRESSION.md
 - DESKTOP_APP_PLAN.md
 - DOC_STATUS.md
 - EVALUATION.md
 - EXECUTION_POLICY.md
 - EXPERIMENT_HARNESS.md
 - GOLDEN_REGRESSION_FIXTURES.md

- HOSTING_PLAN.md
- HOW_RALLY_DETECTION_WORKS.md
- I18N_PLAN.md
- KHELSUTRA_PRODUCT_OVERVIEW.md
- mockups/
 - ui_alpha_mockup.html
- MULTI_SIGNAL_FUSION_PLAN.md
- NEXT_STEPS.md
- NIGHTLY_ENGINE_REGRESSION.md
- OWNED_MODEL_IMPLEMENTATION_PLAN.md
- PACKAGING_PLAN.md
- PER_VIDEO_WORKSPACE.md
- PERSONALIZATION_PLAN.md
- PLATFORM_ARCHITECTURE.md
- POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
- PROJECT_DOC_TEMPLATE.md
- PROMINENT_COURT_DETECTION.md
- QUALITY_ITERATIONS.md
- R1_MILESTONE_LAUNCH_REPORT.md
- R2_EVAL_METRIC_DESIGN.md
- RALLY_DETECTION_CODE_REVIEW.md
- RALLY_DETECTION_FIXES_PLAN.md
- RALLY_DETECTION_GAP_CLOSURE.md
- RALLY_DETECTION_QUALITY_REPORT.md
- RALLY_QUALITY_RESEARCH.md
- README.md
- REAL_FOOTAGE_VALIDATION_REPORT.md
- ROADMAP.md
- SETUP_NEW_MACHINE.md
- STORAGE_SHARING_MODEL.md
- TRACKNET_WSL_SETUP.md
- UI_ALPHA_EXECUTION_PLAN.md
- UI_PROPOSAL.md
- VIDEO_LOCALITY_MODEL.md
- eval_baselines/
 - experiment_leaderboard.json
 - fixtures/
 - [38 files in subtree: 19 *.json, 18 *.csv, 1 *.md]
 - gemini_wrapper_2026-06-10.json
 - heuristic_lovo_n15.json
 - heuristic_lovo_n6.json
 - nightly_baseline.json
 - reelcount_manifest.json
- mypy.ini
- output/
 - chunks/
 - p2b_golden/
 - [53 files in subtree: 30 *.csv, 19 *.json, 2 *.py, ...]
 - reports/
 - [6 files in subtree: 4 *.md, 2 *.json]
- pyproject.toml
- pytest.ini
- README.md
- requirements-trackers.txt
- requirements.txt
- ruff.toml
- run_all_tests.py
- run_eval.py
- run_regression.py
- scripts/
 - __pycache__/
 - [6 files in subtree: 6 *.pyc]
 - bundle_ui.sh
 - doctor.py
 - install.py

- local_ci.sh
- nightly_email.py
- nightly_reelcount.py
- nightly_regression.ps1
- nightly_regression.py
- pre-push
- README.md
- register_nightly_task.ps1
- run_phase3_eval.py
- run_process_and_stitch.py
- uninstall.py
- verify_compute_path.py
- SESSION_HANDOFF.md
- tests/
 - [308 files in subtree: 156 *.pyc, 152 *.py]
- tools/
 - __init__.py
 - __pycache__/
 - [6 files in subtree: 6 *.pyc]
 - check_md_links.py
 - cleanup_caches.py
 - generate_highlights.py
 - leak_scan.py
 - upload_retention.py
- training/
 - __init__.py
 - __pycache__/
 - __init__.cpython-312.pyc
 - gen0/
 - [4 files in subtree: 2 *.py, 2 *.pyc]
 - README.md
 - requirements-train.txt
- f_drive_index/
 - build_outputs.py
 - catalog_all_files.nul
 - catalog_raw.tsv
 - classify.err
 - classify.py
 - confirm_groups.json
 - dedup_report.md
 - dedup_stage1.py
 - dedup_stage2.py
 - ffprobe_meta.json
 - ffprobe_meta.py
 - finalize.out
 - full_md5.json
 - fullmd5.py
 - fullmd5_confirm.out
 - gcs_full_ls.txt
 - gcs_migration_gap.md
 - gcs_videos_ls.txt
 - golden_manifest.json
 - golden_map.json
 - master_video_index.json
 - partial_sigs.json
 - progress.log
 - size_groups.json
 - stage2.err
 - STORAGE_MAP_PROPOSAL.md
 - videos.json
- khelsutra/
 - deploy/
 - cloudflared.config.example.yml
 - DEPLOY_ALPHA.md
 - droplet/

- [4 files in subtree: 2 *.example, 1 *.md, 1 *.service]
 - GPU_VM_SERVING.md
 - stage_from_bucket.sh
- docs/
 - ANNOTATE_AND_TRAIN.md
 - archives/
 - FRIENDLY_AND_MULTILINGUAL.md
 - DRY_RUN_CUJ.md
 - LOCAL_APPLIANCE_ARCHITECTURE.md
 - MULTILINGUAL_EXPANSION.md
 - PER_RALLY_SELECTION.md
 - PORTABILITY.md
 - README.md
 - REQUEST_ACCESS_FORM.md
 - STUDIO_MEDIA_LIFECYCLE.md
- locales/
 - da/
 - common.json
 - en/
 - common.json
 - es/
 - common.json
 - hi/
 - common.json
 - id/
 - common.json
 - kn/
 - common.json
- NEXT_STEPS.md
- README.md
- site/
 - make_og.py
 - package-lock.json
 - package.json
 - public/
 - [30 files in subtree: 11 *.html, 9 *.png, 6 *.json, ...]
 - README.md
 - schema.sql
 - scripts/
 - copy-locales.mjs
 - droplet-smoke.sh
 - provision-vm.sh
 - teardown-vm.sh
 - server.js
 - src/
 - [13 files in subtree: 10 *.js, 3 *.mjs]
 - wrangler.toml
- web/
 - e2e/
 - [11 files in subtree: 9 *.mjs, 1 *.md, 1 *.mp4]
 - index.html
 - package-lock.json
 - package.json
 - README.md
 - src/
 - [66 files in subtree: 34 *.ts, 30 *.tsx, 1 *.css, ...]
 - tsconfig.json
 - vite.config.ts
- KheISutraGuru.jpeg
- marketing/
 - AUDIENCE-TARGETS.md
 - CHANNEL-PLAYBOOKS.md
 - CODEMAP.md
 - CONTENT-CALENDAR.md
 - CONTENT-ENGINE.md

- content/
 - published/
 - queue/
 - [13 files in subtree: 13 *.md]
 - scheduled/
- decisions/
 - [4 files in subtree: 4 *.md]
- docs/
 - [7 files in subtree: 5 *.md, 1 *.html, 1 *.pdf]
- GO-LIVE.md
- GOVERNANCE.md
- LOG.md
- PICKUP-BRIEF.md
- README.md
- REGION-PLAYBOOKS.md
- SETUP-CHANNELS.md
- SETUP.md
- STRATEGY.md
- rally-annotator/
 - CHANGELOG.md
 - CONTRIBUTING.md
 - docs/
 - CSV_FORMAT.md
 - ENDING_REASONS.md
 - LOCALIZATION.md
 - LICENSE
 - README.md
 - test/
 - dialog_layout.snapshot
 - dialog_test.lua
 - README.md
 - vlc/
 - rally_annotator.lua
 - web/
 - [10520 files in subtree: 4215 *.js, 2509 *.ts, 905 *.map, ...]
- rally-corpus-vault/
 - LICENSE
 - pyproject.toml
 - README.md
 - schema/
 - corpus_index.schema.json
 - seed/
 - [4 files in subtree: 2 *.md, 1 *.json, 1 *.tsv]
 - STORAGE_MAP.md
 - storagectl/
 - [4 files in subtree: 2 *.py, 2 *.pyc]
 - tests/
 - [6 files in subtree: 4 *.py, 2 *.pyc]
 - vault.example.json
- sports-data-collector/
 - cleanup.py
 - config/
 - config.yaml
 - data/
 - [13 files in subtree: 6 *.csv, 3 *.json, 2 *.md, ...]
 - docs/
 - [11 files in subtree: 10 *.md, 1 *.csv]
 - pyproject.toml
 - README.md
 - src/
 - [26 files in subtree: 25 *.py, 1 *.yaml]
 - tests/
 - [26 files in subtree: 26 *.py]
- sports-obsreport/
 - LICENSE

- pyproject.toml
- README.md
- schema/
 - report_envelope.schema.json
- scripts/
 - gen_schema.py
- specs/
 - example.yaml
- src/
 - obsreport/
 - [22 files in subtree: 11 *.py, 11 *.pyc]
- tests/
 - [10 files in subtree: 10 *.py]

9. tool_result

```
1?---
name: review
description: >-
  Run a reviewer subagent against uncommitted local changes, a named branch,
  or a GitHub PR. Local and branch modes write a review file plus a summary to disk.
  PR mode posts the findings as a PENDING GitHub review for the user to inspect and
  submit through the UI.
when-to-use: "Use when asked to 'review', 'code review', 'review my changes', 'review this PR',
or '/review'."
argument-hint: "[--local | --branch <name> | --pr <number-or-url> | <auto-detect>]"
10?---
```

Review Skill

You are an orchestrator that runs a reviewer subagent against one of three review targets. You coordinate only ? **all** review findings are authored by a subagent whose prompt is seeded with the `reviewer` persona instructions, never by the orchestrator directly.

Persona Injection

This skill uses the **reviewer** persona. The persona instructions are defined at:

```
20?```
<dirname of this SKILL.md>../shared/personas/reviewer.md
```
```

Resolve this path once at the start of the run (the system context gives you the absolute path to this SKILL.md). Read the file with `read\_file` and store its contents as `reviewer\_persona\_instructions`.

When launching the reviewer subagent, **prepend** the persona instructions to the prompt. Do NOT pass a `persona` parameter to `spawn\_subagent` ? that parameter is not supported. Instead, prefix the `description` with `[reviewer]` so the pager's subagent label renderer surfaces "Reviewer" at the top of the subagent row (see Step 2 below).

1. **Local mode (default)** -- uncommitted local changes (staged + unstaged + untracked).
2. **Branch mode** -- the diff between a named branch and its merge-base with the default base branch.
3. **PR mode** -- a GitHub pull request. Findings are posted as a PENDING review for the user to inspect and submit through GitHub.

The reviewer subagent is read-only -- it never modifies code. The orchestrator never edits source either; the only artifacts produced are a review file, a summary file, and (in PR mode) a pending GitHub review.

## Invocation

The user runs:

```
...
/review # local mode (default)
40?/review --local # local mode (explicit)
/review --branch <name> # branch mode (explicit)
/review --pr <number-or-url> # PR mode (explicit)
/review <plain-arg> # auto-detect; see disambiguation below
...
```

## Argument parsing

Parse the argument string with these deterministic rules, applied in order. The first rule that matches wins; do not fall through.

1. **Empty / whitespace-only**: `MODE=local`, no target.
2. **Starts with `--local`**: `MODE=local`. Reject any extra positional argument with an error.
3. **Starts with `--branch <name>`**: `MODE=branch`, `TARGET=<name>`. The branch name is required -- if the flag appears with no following token (or only with another `--`-prefixed token), reject with `Flag --branch requires an argument: <branch-name>` and stop.
4. **Starts with `--pr <id-or-url>`**: `MODE=pr`, `TARGET=<id-or-url>`. The id or URL is required -- if the flag appears with no following token (or only with another `--`-prefixed token), reject with `Flag --pr requires an argument: <number-or-url>` and stop.
5. **Starts with `--` but does not match any of the above**: reject with `Unknown flag: <flag>`. Valid flags: `--local`, `--branch <name>`, `--pr <number-or-url>`. and stop. Do NOT fall through to auto-detect.
6. **Plain argument given (no flag prefix)**: auto-detect against the rules below, also applied in order:
  1. Matches the regex `^https?://github\.com/[^\s]+/[^\s]+/pull/\d+(?:[/\#].*)?$` -- treat as PR URL. `MODE=pr`, `TARGET=<url>`.
  2. Matches `^#\d+$` (optional leading `#`, then pure digits) -- treat as PR number. `MODE=pr`, `TARGET=<digits without leading #>`.
  3. Resolves to a local or remote branch via `git rev-parse --verify --quiet <arg>` or `git rev-parse --verify --quiet origin/<arg>` -- treat as branch. `MODE=branch`, `TARGET=<arg>` (use the bare name, not `origin/<arg>`).
  4. None of the above -- ask the user whether the argument is a PR identifier or a branch name (use the appropriate ask/question tool if available). Provide three options: "PR (treat as PR identifier)", "Branch (treat as branch name)", and "Cancel". On Cancel, stop.

60?

If the user passes both a flag and a positional argument (e.g., `/review --local somebranch`), reject with a clear error message and stop. The flags are mutually exclusive.

## Setup

Generate a unique ID for this run's artifact files. Execute this via `run_terminal_cmd` and capture stdout:

```
```bash
python3 -c "import uuid; print(uuid.uuid4().hex[:8])"
```
```

70?

This matches the pattern used by `implement/SKILL.md`. The previous draft of this skill chained two fallbacks (`/proc/sys/kernel/random/uuid` and `date +%s`), but the `/proc` path is missing on macOS and the `date` fallback's `tail -c 9` kept a trailing newline -- both bugs. `python3` is reliably present in the supported environments; if it is genuinely absent, the validation step below catches it with a clear error.

**Validate** that the command produced a non-empty 8-character string. If `REVIEW_ID` is empty or the command failed, report the error to the user (with the suggestion to install Python 3) and stop -- do not proceed with empty/malformed file paths.

Store the output as `REVIEW_ID`. Set a restrictive umask first so all subsequent artifact writes land at mode 0600 -- the diff and review files can capture `.env` snippets or other secrets and the default 0644 leaks them to other users on shared hosts:

```
```bash
umask 077
```
```

80?

Then define the file paths used throughout the run:

```
- `summary_file`: `/tmp/grok-review-summary-${REVIEW_ID}.md` (orchestrator-written; used in
local and branch modes only -- not written or read in PR mode)
- `review_file`: `/tmp/grok-review-${REVIEW_ID}.md` (reviewer subagent writes here; produced in
all modes)
- `diff_file`: `/tmp/grok-review-diff-${REVIEW_ID}.diff` (collected diff fed to the reviewer;
produced in all modes)
- `pending_review_payload`: `/tmp/grok-review-pending-${REVIEW_ID}.json` (PR mode only -- not
written or read in local/branch modes, so cleanup of this path is a no-op outside PR mode)
```

Initialize state variables:

```
90?- `mode`: one of `local`, `branch`, `pr` (set by argument parsing).
- `target`: the branch name, PR number, or PR URL (empty in local mode).
- `head_sha`, `base_sha`, `owner`, `repo`, `pr_number`, `pr_url`, `pr_title` (populated in PR
mode by Step 1).
- `changed_files`: list of file paths in the diff (populated by Step 1).
```

## Step 1: Resolve target & collect diff

The diff collection commands differ per mode.

### Local mode

100?

1. Detect changes:

```
```bash
git status --porcelain
```
```

If the output is empty, print "No local changes to review (working tree clean)." Skip directly to Step 4 (Cleanup -- use the local/branch sub-case; both ``<diff_file>`` and the helper file list will be no-op `rm -f` since neither was written yet) and then Step 5 (Final report). The Final report should use the "Local / branch (empty-diff exit)" bullet. Do NOT launch the reviewer.

2. Build a unified diff covering staged + unstaged tracked changes AND untracked files:

110?

```
```bash
# Staged + unstaged tracked changes (includes deletions and modifications).
# Guard against fresh `git init` repos with no commits -- `git diff HEAD`
# fails there with "ambiguous argument 'HEAD'". In that case, leave the
# tracked-change portion empty and let the untracked loop populate the file.
# `core.quotePath=false` keeps non-ASCII / space-bearing paths unquoted so
# the parser in Step 3 PR mode sees literal paths.
if git rev-parse --verify --quiet HEAD >/dev/null; then
    git -c core.quotePath=false diff HEAD > "${diff_file}"
120? else
    : > "${diff_file}"
fi

# Append each untracked file as an added-file diff (skip ignored files)
git ls-files --others --exclude-standard -z | while IFS= read -r -d '' f; do
    git -c core.quotePath=false diff --no-index -- /dev/null "$f" >> "${diff_file}" || true
done

# Size check: print the byte count so the orchestrator can gate continuation.
130? # See the executable size check handling immediately below.
wc -c "${diff_file}"
```

...

Trade-off note: ``git diff --no-index`` exits with status 1 when differences are present (which is always, here), so we suppress its exit with ``|| true``. The alternative `--`git add -N <untracked>`` && `git diff HEAD`` && `git rm --cached <untracked>`` -- mutates the index and risks leaving the user in an unexpected state if interrupted; the ``--no-index`` approach is non-mutating, which is why this block uses it.

****Fresh-repo guard****: on a brand-new ``git init`` with zero commits there is no ``HEAD`` to diff against, so the ``if git rev-parse --verify --quiet HEAD`` branch above falls through to the ``else`` and ``${diff_file}`` starts empty. Everything in the working tree at that point is untracked from git's perspective, so the subsequent ``git ls-files --others`` loop captures it correctly. The rest of the local-mode flow is unchanged.

****Size gate (orchestrator-side)****: read the byte count emitted by ``wc -c`` and act on it:

- ****> 10 MB****: abort with an error telling the user to add the offending paths to ``.gitignore`` (point them at ``git status --porcelain`` plus ``du -sh`` to find the worst offenders -- typical culprits are an untracked ``.node_modules/``, ``.cache/``, ``target/``, or a stray dataset). Do NOT launch the reviewer; run cleanup and stop.
- 140? - ****> 1 MB****: ask the user to confirm before continuing (use the appropriate ask/question tool if available). On decline, run cleanup and stop. The reviewer subagent has a context limit and a multi-MB diff will saturate it.
- ****otherwise****: proceed silently.

Edge cases to be aware of (these do not break the workflow, but the user should be told if they apply):

- ****Very large untracked files**** (binary blobs, generated artifacts that were never ``.gitignore`-d`) get appended to ``${diff_file}`` in full. The size gate above handles this -- the prose here is just a pointer to the executable check above.
- ****Symlinks**** surfaced by ``git ls-files --others`` produce a single ``Symbolic link`` line in the diff rather than file content; the reviewer can still report on the symlink itself but cannot inspect its target.

3. Capture the changed-files list (same fresh-repo guard as in step 2 -- if there is no ``HEAD``, skip the tracked-name listing and rely entirely on ``git ls-files --others``):

```
```bash
150? {
 if git rev-parse --verify --quiet HEAD >/dev/null; then
 git -c core.quotePath=false diff --name-only HEAD
 fi
 git ls-files --others --exclude-standard
} | sort -u > /tmp/grok-review-files-${REVIEW_ID}.txt
```
```

Read this into ``changed_files``.

160?### Branch mode

1. Determine the base branch. Try in order:

```
```bash
if git rev-parse --verify --quiet origin/main >/dev/null; then
 BASE=origin/main
elif git rev-parse --verify --quiet origin/master >/dev/null; then
 BASE=origin/master
else
170? BASE=""
fi
```
```

Use an explicit ``if/elif/else`` (not two unconditional ``&&`` lines) so that ``origin/master`` does not overwrite ``origin/main`` when both exist (which happens during master-to-main migrations and on mirror repos). The ``>/dev/null`` redirect prevents the SHA emitted by ``git rev-parse`` from leaking into the orchestrator's captured output.

If `BASE` is empty (neither ref exists), ask the user which base ref to compare against (use the appropriate ask/question tool if available) (offer the local default branch name(s) you can detect via `git symbolic-ref refs/remotes/origin/HEAD`, plus an "Other" option).

2. Verify the target branch exists:

```
180?  ```bash
      git rev-parse --verify --quiet "${target}" || git rev-parse --verify --quiet
      "origin/${target}"
      ```
```

If neither resolves, report the error and stop.

3. Compute the merge base and collect the diff (`core.quotePath=false` prevents C-style path quoting so the parser in Step 3 PR mode sees literal paths -- `gh pr diff` does not quote paths, so PR mode is unaffected):

```
```bash
MERGE_BASE=$(git merge-base "${BASE}" "${target}")
190?  git -c core.quotePath=false diff "${MERGE_BASE}".."${target}" > "${diff_file}"
      git -c core.quotePath=false diff --name-only "${MERGE_BASE}".."${target}" > /tmp/grok-review-
      files-${REVIEW_ID}.txt
      ```
```

4. **Empty-diff handling**: if `\${diff\_file}` is empty (or contains only whitespace), print "Branch `\${target}` has no changes vs `\${BASE}`." Skip directly to Step 4 (Cleanup -- use the local/branch sub-case) and then Step 5 (Final report). The Final report should use the "Local / branch (empty-diff exit)" bullet to note that the branch has no changes vs its base. Do NOT launch the reviewer.

Read `changed\_files` from the names file.

## PR mode

200?1. Verify `gh` authentication:

```
```bash
gh auth status
```
```

If this exits non-zero, warn the user that the PR cannot be fetched without `gh` auth, then stop with instructions to run `gh auth login`.

2. Fetch PR metadata in a single round trip (works for both numeric IDs and full URLs). Note that `gh pr view --json` does NOT expose a `baseRepository` field (verified: `gh pr view 1 --json baseRepository` returns `Unknown JSON field: "baseRepository"`). The available repo fields are `headRepository`, `headRepositoryOwner`, and `isCrossRepository`. The owner/repo for the upstream where the PR lives must therefore be derived from the `url` field instead. The `files` field is also requested here so we do not need a second round trip:

```
210? ```bash
 gh pr view "${target}" --json number,title,body,headRefOid,baseRefOid,headRefName,baseRefName
 ,url,headRepository,headRepositoryOwner,isCrossRepository,files \
 > /tmp/grok-review-prmeta-${REVIEW_ID}.json
      ```
```

Parse the JSON and populate:

```
- `pr_number` from `.number`
- `pr_title` from `.title`
- `pr_url` from `.url`
- `head_sha` from `.headRefOid`
```

```
220?  - `base_sha` from `.baseRefOid`
```

```
- `owner`, `repo` -- parse from `pr_url` using the regex
```

^https?://github\.(com|com.cn|gov|mil|net|us)/(?<owner>[^\s/]+)/(?<repo>[^\s/]+)/pull/\d+`. The URL always points at the upstream where the PR lives, even for cross-repo PRs (`isCrossRepository: true`), so this is the

correct source for the review-posting endpoint.

```
- `changed_files` -- extract via `jq -r '.files[].path' /tmp/grok-review-prmeta- $\{$ REVIEW_ID $\}$ .json > /tmp/grok-review-files- $\{$ REVIEW_ID $\}$ .txt`, then read the file.
```

****Validate**** that all required fields are non-empty: `pr\_number`, `head\_sha`, `base\_sha`, `owner`, `repo`. If any is missing or empty (which can happen for very old PRs, PRs in unusual states, or partial responses), surface the parsed JSON to the user and stop -- do not proceed to build a payload with `null` fields.

3. Fetch the diff:

```
```bash
gh pr diff "${target}" > "${diff_file}"
230? ```
```

4. **\*\*Empty-diff handling\*\***: if `\${diff\_file}` is empty, print "PR # $\{$ pr\_number $\}$  has no changes." Skip directly to Step 4 (Cleanup -- use the "PR mode, no reviewer output" sub-case, which removes ``, the prmeta JSON, the files list, and is a no-op on `` / `` because neither was written) and then Step 5 (Final report). The Final report should use the "PR (empty-diff exit)" bullet to note that the PR has no changes. Do NOT launch the reviewer.

After Step 1, report progress: "Collected diff for <mode> target <summary>. Launching reviewer..."

## Step 2: Launch reviewer subagent

Launch a single reviewer subagent by calling `spawn\_subagent`. Emit the `spawn\_subagent` tool call before producing any "reviewer is starting" narration; the post-launch progress message ("Review complete. Processing findings...") belongs in a later assistant message after the tool result is in hand.

240? `spawn\_subagent` parameters:

```
- `subagent_type`: "general-purpose"
- `description`: "[reviewer] <mode> <target-summary>" (e.g., "[reviewer] pr #4221" or "[reviewer] branch feature/foo" or "[reviewer] local changes"). The `[reviewer]` prefix is parsed by the pager's subagent label renderer (see `format_subagent_label` in `xai-grok-pager`) so the subagent row shows "Reviewer" instead of the generic "General" fallback. The bracketed prefix is stripped from the displayed description.
```

Build the prompt with the mode-specific context. **\*\*Prepend the reviewer persona instructions\*\*** (loaded during setup) to the prompt. Use this template:

```
...
<reviewer_persona_instructions>
```

250?---

You are reviewing code changes. Mode: <mode>.

```
Target: <target-summary-line>
<if PR mode: PR URL: <pr_url>>
<if PR mode: head SHA: <head_sha>, base SHA: <base_sha>>
<if branch mode: base: <BASE>, merge-base: <MERGE_BASE>, head: <target>>
```

The unified diff is at: <diff\_file>

260?The list of changed files is at: /tmp/grok-review-files- $\{$ REVIEW\_ID $\}$ .txt

Read the diff first to understand the scope. The diff alone is often not enough context, so you should also `read\_file` the source files referenced in the diff to understand call sites, types, and surrounding logic before flagging issues.

Write your structured findings to: <review\_file>



Format:

270?## Summary

<2 to 4 sentence overall assessment of the changes -- what they do, whether they look correct, the dominant risk areas. This goes at the very top of the file, before any individual issues.>

## Issues

### Issue 1 -- Severity: bug

- File: path/to/file.ext:LINE  
280?- Description: <what is wrong>  
- Suggestion: <how to fix>  
- Status: open

### Issue 2 -- Severity: suggestion

- File: path/to/file.ext:LINE  
- Description: ...  
- Suggestion: ...  
- Status: open

290?Severity must be one of: bug, suggestion, nit. Each issue's Status field must be set to "open" (as shown in the example above).

<if PR mode, include this paragraph verbatim:>

IMPORTANT: For each issue, the File line MUST reference a single line number on the RIGHT side of the diff (the line number in the new/post-change file, not the pre-change file). If a finding spans a range, pick the most representative single line on the RIGHT side. This requirement is mandatory because the orchestrator will post these findings as inline comments on the GitHub PR, and the GitHub API rejects comments that do not target a line present in the diff.  
<end if>

300?

If the diff is genuinely fine and you have no issues, write the Summary and an empty `## Issues` section (or omit the Issues section entirely). Do not invent issues to fill space.  
...

Wait for the subagent to complete. If it fails, report the error to the user and stop.

After it completes, verify that `<review\_file>` exists and is non-empty. If it does not, report the error and stop.

310?Save the returned `subagent\_id` for the report. The reviewer is not resumed; this is a one-shot review.

Report progress: "Review complete. Processing findings..."

## Step 3: Handle output based on mode

The post-processing differs between local/branch mode (write summary to disk) and PR mode (post pending review to GitHub).

### Local / Branch mode

320?1. Read `<review\_file>` via `read\_file`.

2. Count issues by counting heading lines that match the regex `^### Issue \d+ -- Severity: (bug|suggestion|nit)\$` and bucket them by the captured severity. Issues whose heading does not match this pattern (typo'd severity, missing severity field, malformed heading) are malformed -- log a one-line warning to the user listing the heading line and treat them as uncounted. Do not attempt to re-parse the body for a `Severity:` field; the heading is the canonical source. The same regex governs the parsing in Step 3 PR mode below.

### 3. Compute diff stats:

```
```bash
git diff --shortstat ...      # local: HEAD; branch: MERGE_BASE..target
```
```

For local mode, run `git diff --shortstat HEAD` and also count untracked files separately. For branch mode, run `git diff --shortstat "\${MERGE\_BASE}".."\${target}"`.

330?4. Use the `write` tool to create `` with this structure:

```
```markdown
# Review Summary

- **Mode**: <mode>
- **Target**: <target-summary>
- **Files reviewed**: <count> (<list, truncated to 10 if longer>)
- **Diff stats**: <shortstat-line>
- **Issue counts**: <X> bugs, <Y> suggestions, <Z> nits

340?
## Top issues

<First 5 issue titles, one per line, in the form "[severity] file:line -- description
(truncated to ~100 chars)">

See the full review at: <review_file>
```
```

### 5. Print to the user:

- Inline issue counts and the file paths to `` and ``.

350?

Do NOT delete `` or `` in local/branch modes -- those files ARE the deliverable.

## PR mode

### 1. Read `` via `read\_file` and parse it into a structured representation:

- Extract the `## Summary` section -- everything after the `## Summary` heading and before the next `##` heading.
- Extract each issue by matching heading lines against `^### Issue \d+ -- Severity: (bug|suggestion|nit)\$` (same regex as Step 3 Local/Branch mode). For each matched block, capture:
  - `severity` -- the captured group from the heading regex
  - `file` and `line` -- parsed from the `File: path:line` field. If `:line` is missing, the issue cannot become an inline comment; it must be promoted to the body.
  - `description` -- from the `Description:` field
  - `suggestion` -- from the `Suggestion:` field (may be empty)

**\*\*Early-exit on zero issues\*\*:** if the parsed issue list is empty, do NOT walk the diff, do NOT build a payload, and do NOT post anything to GitHub. Posting an empty PENDING review is wasteful, looks spammy on the PR, and gives the user a "submit your review" reminder for a review with nothing in it. Instead:

- Print: "Reviewer found no issues on PR #\${pr\_number}. No PENDING review created."
- Skip directly to Step 4 (Cleanup) and then Step 5 (Final report). The Final report should note that no PENDING review was posted.

### 2. Read `` via `read\_file` and walk it to determine which `(file, line)` pairs are present on the RIGHT side of any hunk. The line is on the RIGHT side when:

- It is an added line (prefix `+`, but not the `+++ b/...` header line), OR
- It is a context line (prefix ` `).

370?

Walk the diff file by file:

- Each file section starts with `+++ b/<path>` (treat `+++ /dev/null` as a deletion -- skip).
- Within a file, each hunk starts with `@@ -<old>[,<oldcount>] +<new>[,<newcount>] @@`. The `,<count>` parts are optional and default to `1` when omitted (so `@@ -42 +42 @@` is a valid

single-line hunk that ``git diff`` and ``gh pr diff`` both emit). A regex like ``^@@ -(\d+)(?:,\d+)? \+(\d+)(?:,\d+)? @@`` matches both forms; capture ``<new>`` from the second group. Reset the right-side line counter to ``<new>``. (The regex is intentionally unanchored at the end -- real diffs frequently carry trailing context after the second ``@@``, e.g. ``@@ -10,5 +15,3 @@ def my_function():``, and ``re.match`` succeeds on the prefix without a ``$`` anchor. Do NOT add ``$``; it would reject every hunk header that includes a function/class hint.)

- Walk the hunk body. For `` `` (context) and ``+`` lines (excluding ``+++`` headers), record the current right-side line and increment the counter. For ``-`` lines (excluding ``---`` headers), do not increment the right-side counter.

- Lines starting with ``\ `` (a literal backslash followed by a space) are diff metadata, not file content -- e.g., ``\ No newline at end of file``. Skip them: do not increment the right-side counter, and do not contribute a ``(file, line)`` pair.

Build a set of ``(file, line)`` pairs.

3. Partition the parsed issues into two groups:

380? - **Inline comments**: issues whose ``(file, line)`` is in the diff set.

- **Promoted to body**: issues whose ``(file, line)`` is missing or not in the diff set.

4. Build the JSON payload via the ``write`` tool, saving to ``<pending_review_payload>``:

```
```json
{
  "commit_id": "<head_sha>",
  "body": "<assembled body, see below>",
  "comments": [
390?   {
      "path": "src/foo.rs",
      "line": 42,
      "side": "RIGHT",
      "body": "***[bug]** description text\n\n**Suggestion:** suggestion text"
    }
  ]
}
```
```

400? **Do NOT include the ``event`` field.** Omitting ``event`` causes the GitHub API to create the review in PENDING state, which is exactly what we want -- the user reviews and submits it through the GitHub UI.

The top-level ``body`` is constructed as:

```
```
## Summary

<verbatim text of the ## Summary section from the review_file>

## Issue counts by severity
410? - bugs: <X>
    - suggestions: <Y>
    - nits: <Z>

<if any promoted issues exist, append:>
## Issues outside the diff
```
```

These findings reference lines that are not present in the diff and could not be posted as inline comments:

```
420? - [severity] file:line -- description
 - Suggestion: suggestion text
 <repeat for each promoted issue>
 <end if>
```
```

Each inline comment body has the form `***[severity]** description\n\n**Suggestion:** suggestion``. If the suggestion is empty, drop the suggestion line.

`**Construct the payload as a Python dict, then serialize via json.dumps.` Do NOT concatenate JSON strings by hand: review text routinely contains ```, `\\``, or embedded newlines, and naive concatenation produces invalid JSON which `gh api`` rejects with 400/422. Run a short `python3`` heredoc to materialize the JSON string and persist it to `<pending_review_payload>` through the `write`` tool:

```
430?  ```bash
python3 <<'PY'
import json
payload = {
    "commit_id": "<head_sha>",
    "body": "<assembled body>",
    "comments": [
        {
            "path": "src/foo.rs",
            "line": 42,
440?         "side": "RIGHT",
            "body": "***[bug]** description text\n\n**Suggestion:** suggestion text",
        },
        # ... one entry per inline comment
    ],
}
print(json.dumps(payload, ensure_ascii=False, indent=2))
PY
...`
```

450? `json.dumps(payload, ensure_ascii=False, indent=2)`` handles all quote, backslash, and newline escaping automatically. Capture the printed JSON and pass it as the `content`` argument to the `write`` tool with `<pending_review_payload>` as the file path.

5. Post the review:

```
```bash
gh api "repos/{owner}/{repo}/pulls/{pr_number}/reviews" \
-X POST \
--input "${pending_review_payload}" \
> /tmp/grok-review-post-${REVIEW_ID}.json 2> /tmp/grok-review-post-${REVIEW_ID}.err
...`
```

460? Capture both stdout and stderr.

6. **Error handling**: if the `gh api`` call exits non-zero, surface the captured stderr verbatim to the user along with the HTTP status code (parseable from `gh api``'s stderr message). Do not retry. Common cases by status:

- **422 Unprocessable Entity**: comments reference lines outside the diff (the diff-line filter missed something, e.g., the reviewer hallucinated a file path), `commit_id`` is stale (the PR was force-pushed between Step 1 and now), or the `side`` value was rejected.
- **403 Forbidden**: authenticated user lacks permission to comment on the PR (read-only collaborator on the repo, archived repo).
- **404 Not Found**: PR does not exist (the user passed a wrong number, or the URL is in a private repo the user cannot see).
- **5xx**: GitHub-side outage or transient failure.

On any error, also keep `<review_file>` on disk (skip the PR-mode review\_file deletion in Step 4) so the user can see what would have been posted and either re-run with corrected inputs or submit the notes through some other channel. Mention the preserved path in the error message.

470? 7. On success, the user-facing pending-review URL is the PR Files tab, where pending reviews surface and where the "Finish your review" / "Submit review" button lives:

```
...`
https://github.com/<owner>/<repo>/pull/<pr_number>/files`
```

...

Construct this URL from the already-validated `owner`, `repo`, and `pr\_number` -- do NOT use the `html\_url` field from the GitHub response. The response's `html\_url` points at a deep link to the review object that does not surface the submit button as cleanly as the Files tab. (The response is still parsed for the review `id`, which is recorded in the Final report; the `html\_url` field is not used.)

Print to the user as a structured block (not a wall of text):

480?

...

A PENDING review has been created on PR #<pr\_number>.

- Inline comments: <N>
- Body findings (outside the diff): <M>
- Submit at: [https://github.com/<owner>/<repo>/pull/<pr\\_number>/files](https://github.com/<owner>/<repo>/pull/<pr_number>/files)  
(scroll to "Finish your review" -> click "Submit review").

Until you submit, the comments are visible only to you.

...

490?## Step 4: Cleanup

The cleanup behavior is **asymmetric** by mode and by outcome:

- **Local and branch modes**: keep ``<review_file>`` and ``<summary_file>`` -- they are the deliverable. Remove only ``<diff_file>`` and the ``/tmp/grok-review-files-${REVIEW_ID}.txt`` helper file.
- **PR mode, successful post**: remove ``<review_file>``, ``<diff_file>``, ``<pending_review_payload>``, ``/tmp/grok-review-prmeta-${REVIEW_ID}.json``, ``/tmp/grok-review-files-${REVIEW_ID}.txt``, and the post stdout/stderr capture files. The pending review on GitHub is the deliverable; the local files are no longer needed. ``<summary_file>`` was never written in PR mode, so no action there.
- **PR mode, failed post (any non-zero `gh api` exit)**: keep ``<review_file>`` so the user can see what would have been posted and submit it through some other channel. Remove the rest as in the success path. The Final report must mention the preserved path.
- **PR mode, no reviewer output (covers both zero-issue early-exit AND PR-with-empty-diff exit)**: remove all of ``<diff_file>``, ``<pending_review_payload>`` (never created -- `rm -f` is a no-op), ``/tmp/grok-review-prmeta-${REVIEW_ID}.json``, ``/tmp/grok-review-files-${REVIEW_ID}.txt``, and ``<review_file>`` (which is a no-op in the empty-diff case where the reviewer never ran, and removes the empty-of-actionable-content file in the zero-issue case). This sub-case is the catch-all for any PR-mode exit that did not POST to GitHub.

Cleanup commands:

500?

```bash

Local / branch mode (always -- covers both successful-review and empty-diff-exit cases; the empty-diff exit happens before `<diff_file>` contains anything useful, but `rm -f` is a no-op on a non-existent or already-empty file)

```
rm -f "${diff_file}" /tmp/grok-review-files-${REVIEW_ID}.txt
```

PR mode, successful post

```
rm -f "${review_file}" "${diff_file}" "${pending_review_payload}" \  
    /tmp/grok-review-prmeta-${REVIEW_ID}.json \  
510?    /tmp/grok-review-files-${REVIEW_ID}.txt \  
    /tmp/grok-review-post-${REVIEW_ID}.json \  
    /tmp/grok-review-post-${REVIEW_ID}.err
```

PR mode, failed post -- omit `"${review_file}"` from the rm command

```
rm -f "${diff_file}" "${pending_review_payload}" \  
    /tmp/grok-review-prmeta-${REVIEW_ID}.json \  
    /tmp/grok-review-files-${REVIEW_ID}.txt \  
    /tmp/grok-review-post-${REVIEW_ID}.json \  
    /tmp/grok-review-post-${REVIEW_ID}.err
```

```
/tmp/grok-review-post- $\{REVIEW\_ID\}$ .err
```

520?

PR mode, no reviewer output (zero-issue early-exit OR empty-diff exit)

```
rm -f "${review_file}" "${diff_file}" "${pending_review_payload}" \  
    /tmp/grok-review-prmeta- $\{REVIEW\_ID\}$ .json \  
    /tmp/grok-review-files- $\{REVIEW\_ID\}$ .txt  
...
```

Step 5: Final report

Present a final report to the user. Item 1 is always included; item 2 is omitted on empty-diff exits; item 3 is omitted on empty-diff exits and on PR zero-issue early-exits; item 4 is always included.

530?

1. **Mode + target** -- e.g., "Local changes" or "Branch feature/foo vs origin/main" or "PR #4221: <pr_title>".
2. **Files reviewed** -- count and (if 10 or fewer) the list. Omit on empty-diff exits.
3. **Issues by severity** -- the bug/suggestion/nit counts. Omit on empty-diff exits and on PR zero-issue early-exits (counts are all zero by construction; the mode-specific bullet says so).
4. **Mode-specific output**:
 - Local / branch (successful review): full paths to <review_file> and <summary_file>, plus a one-line copy of the summary's top-issues list.
 - Local / branch (empty-diff exit): "No changes to review." (local mode) or "Branch <target> has no changes vs <BASE>." (branch mode). No file paths -- nothing was written.
 - PR (successful post): the structured submit block from Step 3 PR mode item 7 (PR number, inline-comment count, body-findings count, Files-tab URL, submit reminder), plus the review 'id' from the response (for traceability).
 - PR (zero-issue early-exit): "Reviewer found no issues on PR #<pr_number>. No PENDING review created."
 - PR (empty-diff exit): "PR #<pr_number> has no changes. Nothing to review."

540? - PR (failed post): the verbatim stderr from `gh api` plus the HTTP status code, plus the preserved path to <review_file> so the user can recover the findings.

In-Progress Reporting

Give the user a brief status update after each phase:

- After argument parsing: "Reviewing <mode> target: <target-summary>." (omit target for local mode -- "Reviewing local changes.")
- After Step 1 (diff collection): "Collected diff (<N> files, <M> changed lines). Launching reviewer..."
- After Step 1 (empty diff): "No changes to review. Proceeding to final report."
- After Step 2 (reviewer complete): "Review complete. Processing findings..."
- 550?- After Step 3 (local / branch): "Found N issues (X bugs, Y suggestions, Z nits). Wrote <review_file> and <summary_file>."
- After Step 3 (PR, zero issues): "Reviewer found no issues on PR #<pr_number>. No PENDING review created."
- After Step 3 (PR, success): "Posted PENDING review with N inline comments and M body findings. Visit <url> to submit."
- After Step 3 (PR, error -- 422 or any other non-zero exit): "Failed to post pending review. HTTP <status>. GitHub returned: <verbatim stderr>. Review notes preserved at <review_file>."

Rules

- **Reviewer is read-only** -- the reviewer subagent must never modify files. The orchestrator must never modify source files either. The only writes are to <review_file>, <summary_file>, <diff_file>, <pending_review_payload>, and the GitHub PENDING review.
- **Inject the reviewer persona into the prompt** -- always prepend the <reviewer> persona instructions (from the shared personas file) to the subagent prompt. Do NOT pass a <persona> parameter to <spawn_subagent>.
- **Prefix <description> with <reviewer>** -- the pager's subagent label renderer parses the first <tag> of the description and uses it as the row label (the bracketed prefix is stripped from the displayed description). Without it, the row falls through to the generic "General"

label because `subagent\_type` is `general-purpose` and no persona/role is plumbed through the spawn args.

560?- ****Include context in the reviewer prompt**** -- the reviewer needs the conversation context (the user's framing, any constraints, etc.) passed through the task prompt.

- ****One reviewer per run**** -- this skill runs a single reviewer. Multi-reviewer runs are the job of `/implement` with `--effort N`.
- ****Disambiguation is deterministic**** -- the auto-detect rules in argument parsing are applied in order. Do not second-guess them. If the rules do not match, ask the user; do not guess.
- ****Empty diffs short-circuit**** -- never launch the reviewer with an empty diff. Report "no changes", run cleanup, and produce a Final report (using the appropriate "empty-diff exit" bullet).
- ****Zero issues short-circuits PR mode**** -- if the reviewer finds nothing on a PR, do not post an empty PENDING review. Print a "no issues found" message, clean up, and stop.
- ****PR mode requires `gh` auth**** -- if `gh auth status` fails, stop with a clear instruction to run `gh auth login`. Do not try to work around it.
- ****PENDING reviews are the user's to submit**** -- the orchestrator never sets the `event` field on the review payload. The user reviews and submits via the GitHub UI. Always print the URL plus the submit-via-UI reminder.
- ****Filter inline comments to lines actually in the diff**** -- GitHub returns 422 for comments on lines outside the diff. The orchestrator parses the diff itself to validate `(file, line)` pairs and promotes any out-of-diff issues to the top-level body as bullet points.
- ****Surface every `gh api` error verbatim**** -- if the GitHub API call exits non-zero (422, 403, 404, 5xx, anything else), do not retry. Show the user the raw stderr plus the HTTP status code so they can debug, and preserve `` on disk so the findings are not lost.
- ****No helper script**** -- the orchestrator does the diff parsing and JSON building inline.

570?- ****Cleanup is asymmetric by mode and outcome**** -- local/branch modes keep `` and `` (they are the deliverable). PR mode removes `` on a successful post (the pending GitHub review is the deliverable) and on a no-reviewer-output exit (zero issues OR empty diff -- nothing actionable to preserve), but preserves it on a failed post so the user can recover the findings.

- ****Thread the same file paths**** across all steps -- never regenerate paths between steps. The `REVIEW\_ID` is fixed for the run.
- ****Error handling**** -- if the reviewer subagent fails, the diff collection fails, or the GitHub API call fails, report the error to the user and stop. Do not silently continue with missing results.
- ****No emojis in output**** -- match the conventions of the `reviewer` persona instructions and the surrounding skills.

Design notes

These are background notes that explain the rationale behind some of the choices above. They are NOT rules; they are pointers for future maintainers.

- ****No helper script.**** The markdown parsing of `` and the unified-diff hunk walk are well-defined and small enough to do directly in the orchestrator (using `read\_file` to load the inputs and `write` to create the JSON payload). This follows the same self-contained pattern as `xai-pr-comments`. If a future maintainer finds the inline approach genuinely intractable -- e.g., the parser is consistently miscounting hunks across many real-world PRs -- a helper at `grok/skills/review/scripts/build\_pending\_review.py` could be added with a clear justification. As of this writing, no such helper exists.

580?- ****Owner/repo are derived from the PR `url`, not from a `baseRepository` field.**** `gh pr view --json baseRepository` returns `Unknown JSON field: "baseRepository"` -- only `headRepository`, `headRepositoryOwner`, and `isCrossRepository` are exposed. Parsing the URL is the simplest universal approach and works correctly for cross-repo PRs.

- ****The constructed `/files` URL is used in user output, not the response's `html\_url`.**** The Files tab is where the "Finish your review" / "Submit review" button surfaces in the GitHub UI; that is what the user actually needs to interact with. The response's `html\_url` deep-links to the review object and is less useful for this workflow.

10. assistant

Reviewing PR #550 on `Khelsutra/badminton-highlight-indexer`. Loading the reviewer persona and collecting PR metadata.

11. tool_result

Error: /home/avidullu/.grok/bundled/shared/personas/reviewer.md does not exist.
Note: your current working directory is /home/avidullu/projects/khelsutra-guru

12. tool_result

```
exit: 1
8ad8ff23
github.com
  ? Logged in to github.com account avidullu (/home/avidullu/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_*****
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
Unknown JSON field: "baseRef0id"
Available fields:
additions
assignees
author
autoMergeRequest
baseRefName
body
changedFiles
closed
closedAt
comments
commits
createdAt
deletions
files
headRefName
headRef0id
headRepository
headRepositoryOwner
id
isCrossRepository
isDraft
labels
latestReviews
maintainerCanModify
mergeCommit
mergeStateStatus
mergeable
mergedAt
mergedBy
milestone
number
potentialMergeCommit
projectCards
projectItems
reactionGroups
reviewDecision
reviewRequests
reviews
state
statusCheckRollup
title
updatedAt
url
```

13. tool_result

```
exit: 0
/home/avidullu/.grok/bundled/skills/shared/personas/reviewer.md
```


agents
manifest.json
personas
roles
skills

14. tool_result

```
exit: 0
{"additions":450,"baseRefName":"master","body":"## Why\n\nThree linked observability gaps on
`backend/worker/___main__.py`'s infer path, surfaced during a **live production CUJ (2026-07-09,
GX010155)**. On the cloud-serving path `phase_placement.validation/segment/compile` are all
`cloud_run_l4`, so the control-plane dispatches to the L4 worker **without** `--skip-validation`
? the worker really does run the full validator suite and a ~10-min segmentation. None of it was
observable, so users reasonably concluded validation had been skipped and the ~10-min GPU pass
was a black hole.\n\n## What\n\n### GAP 1 ? validation invisible on a PASS\nThe worker only
`print()`d on SKIP/FAIL; a PASS logged nothing, and the verdicts lived only in the ephemeral
worker DB, never in the pushed `report.json`. Now:\n- `[worker] validating (N checks)?`\n-
`[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`\n- the per-validator verdicts
(`name/passed/message/details`) are folded into `report.json` under `validation`.\n\n### GAP 2 ?
no per-stage wall-clock\nchecksum` / `validation` / `segmentation` are each timed with
`time.perf_counter()`, logged (`[worker] checksum: ?`, `[worker] segmentation (<seg>) done in Zs
? R rallies`), and persisted as a `timings` block in `report.json`. Built incrementally, so a
mid-run failure still records the stages that ran.\n\n### GAP 3 ? the #534 live heartbeat never
fired on the native-L4 path\nTwo compounding bugs:\n1. **Emission:** `wasb_infer.py` never
configured logging, so its periodic `done/total (pct%)` progress went through `logger.info()`
while root stayed at `WARNING` ? the lines were dropped entirely.\n2. **Capture:** even when
emitted they land on **stderr**, but `NativeWasbRunner` streamed only **stdout** (and draining
stdout while the child fills its ~64 KB stderr pipe risks a deadlock).\n\nFix: `wasb_infer`
configures INFO logging to **stdout** at entry (`_configure_logging`), and the runner **merges
stderr?stdout** (line-buf
fered) with a bounded tail for the failure message. The detector progress callback now fires, so
`run_telemetry.jsonl` gets real GPU/velocity snapshots, the `[worker] progress:` heartbeat
appears in stdout, and `
```

stage timing. checksum / validation / segmentation are now each\ntimed with time.perf_counter(), logged (\n"[worker] checksum: ?\n", \n"[worker]\nsegmentation (<seg>) done in Zs ? R rallies\n"), and persisted as a `timings`\nblock in report.json. Built incrementally so a mid-run failure still records\nthe stages that ran.\n\nGAP 3 ? the #534 live heartbeat never fired on the native-L4 path. Two\ncompounding bugs: (a) wasb_infer.py never configured logging, so its periodic\n\ndone/total (pct%)\n progress went through logger.info() while root stayed at\nWARNING ? the lines were dropped entirely; (b) even when emitted they land on\nstderr, but NativeWasbRunner streamed only stdout (and draining stdout while the\nchild fills its ~64 KB stderr pipe risks a deadlock). Fix: wasb_infer configures\nINFO logging to stdout at entry, and the runner merges stderr?stdout\n(line-buffered) with a bounded tail for the failure message. The detector\nprogress callback now fires, so run_telemetry.jsonl gets real GPU/velocity\nsnapshots, the \n"[worker] progress:\n heartbeat appears, and\n<video_id>_detector_progress.json is written to GCS during the run.\n\nreport.json changes are additive; the control-plane reads it via report.get(...)\nso existing readers are unaffected.\n\nVerified locally (Windows): full suite 1965 passed / 1 skipped, coverage 84.44%\n(floor 70), ruff 0.15.16 + mypy 2.1.0 clean. Offline unit tests added for the\ntiming/report-serialization additions, the _val_token score hint, the\nstderr-merge/line-buffer + failure-tail streaming path, and the wasb_infer\nstdout logging config. The live-L4 heartbeat end-to-end still needs an\nowner-driven CUJ to fully confirm.\n\nCo-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>","messageHeadline":"fix(worker): make the cloud-serving L4 infer path observable","oid":"b3dff78311e36ea43c4e0061a17b199396eec8b9"}], "deletions":46, "files":[{"path":"backend/pipeline/detectors/native_wasb_runner.py", "additions":37, "deletions":20}, {"path":"backend/pipeline/detectors/wasb_infer.py", "additions":22, "deletions":0}, {"path":"backend/worker/__main__.py", "additions":111, "deletions":24}, {"path":"tests/test_native_wasb_runner.py", "additions":108, "deletions":2}, {"path":"tests/test_worker.py", "additions":172, "deletions":0}], "headRefName":"fix/worker-l4-infer-observability", "headRefOid":"b3dff78311e36ea43c4e0061a17b199396eec8b9", "headRepository":{"id":"R_kgDOSg1-fw", "name":"badminton-highlight-indexer"}, "headRepositoryOwner":{"id":"O_kgDOEYa5Fw", "login":"Khelsutra"}, "isCrossRepository":false, "number":550, "title":"fix(worker): make the cloud-serving L4 infer path observable", "url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/550"}

```
{"baseRefName":"master", "headRefOid":"b3dff78311e36ea43c4e0061a17b199396eec8b9"}
origin/main
3249451d5c408f8e20bff03e5d4e4c07f0bd0d4d
{"base":"00e8f92f102283660c25492c103f7665e98810a4", "base_ref":"master", "head":"b3dff78311e36ea43c4e0061a17b199396eec8b9", "head_ref":"fix/worker-l4-infer-observability", "number":550, "title":"fix(worker): make the cloud-serving L4 infer path observable", "url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/550"}
```

15. tool_result

1?You are a meticulous code reviewer. Review code and produce structured review notes in a Markdown file at the path given in the prompt.

Process:

1. Read all relevant code thoroughly
2. Write findings to the specified review notes file
3. Use structured format: severity, file:line, description, suggestion, status

Rules:

- 10?- Check correctness first, style second
- Look for edge cases, error handling gaps, race conditions
- Flag unwrap(), unnecessary clone(), or lock usage
- Be specific: cite file:line for every issue
- Do NOT fix the code yourself
- In your final response, state the file path and summarize the verdict

16. tool_result

```
exit: 0
31219 /tmp/grok-review-diff-8ad8ff23.diff
697 /tmp/grok-review-diff-8ad8ff23.diff
backend/pipeline/detectors/native_wasb_runner.py
backend/pipeline/detectors/wasb_infer.py
backend/worker/__main__.py
```

```
tests/test_native_wasb_runner.py
tests/test_worker.py
base 00e8f92f102283660c25492c103f7665e98810a4
head b3dff78311e36ea43c4e0061a17b199396eec8b9
title fix(worker): make the cloud-serving L4 infer path observable
Traceback (most recent call last):
  File "<string>", line 1, in <module>
KeyError: 'files'
From https://github.com/Khelsutra/badminton-highlight-indexer
* [new ref]          refs/pull/550/head -> pr-550-review
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
```

17. assistant

Collected diff (5 files, ~450 additions). Launching the reviewer.

18. tool_result

```
Subagent started in background.
subagent_id: 019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e
type: general-purpose
description: [reviewer] pr #550
```

Use `get_command_or_subagent_output` with `task_ids=["019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e"]` and `timeout_ms` to wait for results.

19. tool_result

```
=== Task 019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e ===
Command: [subagent:general-purpose] [reviewer] pr #550
Status: completed
Started: 2026-07-09T07:16:27Z
Ended: 2026-07-09T07:22:05Z
Duration: 337.65s
Exit Code: 0
Output File:
```

=== Output ===

Review complete

Structured notes written to `***\tmp\grok-review-8ad8ff23.md***`.

Verdict

`**Approve with notes**` ? the three observability fixes look correct and well-tested.

```
Fix	Assessment
Validation PASS logging + `report.json`	Additive, backward-compatible
(`timings`/`validation` omitted when absent)	
Per-stage timings	Sensible; mid-run `_fail()` still gets partial timings
Heartbeat (`wasb_infer` ? stdout + stderr merge)	Right root-cause fix for silent INFO + pipe
deadlock |
```

Issues filed (5)

1. `**bug**` ? Live-progress `Popen` path never applies `timeout\_sec` (production always passes `progress`)
2. `**suggestion**` ? `n\_checks` uses private `\_validators`
3. `**suggestion**` ? Duplicate validation FAIL log lines
4. `**suggestion**` ? Fetch/pull not included in timings
5. `**nit**` ? `--skip-validation` writes `"validation": []` with no skipped flag

Tests for report shape, token formatting, stderr merge kwargs, failure tail, and logging config are solid. No source files were modified.

```
<subagent_meta>id=019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e, type=general-purpose, tool_calls=53, turns=1, duration_ms=337648</subagent_meta>
```

```
<subagent_result>
subagent_id: 019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e
subagent_type: general-purpose
To continue this subagent's conversation, use resume_from="019f45bb-b9c7-7bc3-b5fc-fc26df04ac9e".
</subagent_result>
```

20. assistant

Review complete. Processing findings and preparing the pending GitHub review.

21. tool_result

1?## Summary

PR #550 correctly closes three real L4 observability gaps: validation PASS is now logged and folded into `report.json`, per-stage wall-clock is recorded, and the native-WASB heartbeat is fixed by emitting `wasb\_infer` INFO progress to stdout plus merging stderr into the runner's stream (eliminating the classic stdout-only drain deadlock). The changes are mostly additive and backward-compatible for `report.json` readers that only check `status` / `segments` / `video\_uri`. Dominant residual risk is that the live-progress `Popen` path (the production path, because the worker always passes `progress`) still never enforces `timeout\_sec`, so a hung GPU child can still wedge the worker until an external kill.

Issues

Issue 1 -- Severity: bug

- File: backend/pipeline/detectors/native_wasb_runner.py:386
- Description: The live-progress branch (entered whenever `progress` is not None) launches the child with `subprocess.Popen` and then blocks on `for line in proc.stdout` / `proc.wait()` with no timeout. `NativeWasbConfig.timeout\_sec` is only applied on the fast path via `\_run()` ? `subprocess.run(..., timeout=...)`. The worker always passes a progress callback (`trajectory\_hybrid` ? `run\_predict(..., progress=progress)`), so the cloud native-L4 path this PR is about never enforces the configured 30?minute kill. The surrounding `except subprocess.TimeoutExpired` is therefore dead code for production streaming, and the new comment?s ?hangs until timeout? framing is misleading. A child that stops producing output (or wedges after closing pipes) can block the worker indefinitely.
- 10?- Suggestion: Enforce `timeout\_sec` on the streaming path as well ? e.g. track `deadline = time.monotonic() + timeout`, use a reader thread/`select` with a short poll, or `proc.wait(timeout=remaining)` in a loop and `kill()` + raise/return on expiry (mirroring `\_run`). Add a unit test that a fake Popen which never closes stdout is killed when `timeout\_sec` is small.
- Status: open

Issue 2 -- Severity: suggestion

- File: backend/worker/__main__.py:359
- Description: `n\_checks = len(getattr(validator\_registry, "\_validators", []))` reaches into a private registry attribute. That works today, but it is not part of the public validator-registry API and will silently report `0 checks` if the internal field is renamed or if validators are stored differently.
- Suggestion: Prefer a public helper (e.g. `len(validator\_registry)` / `list\_registered()` if one exists or is added next to `register`), or log without a count (`[worker] validating?`) to avoid coupling to private state.
- Status: open

Issue 3 -- Severity: suggestion

20?- File: backend/worker/__main__.py:369

- Description: On validation failure the new summary line (`validation FAILED in ? ? name=FAIL(score)`) is always followed by the pre-existing failures print later in `cmd\_infer` (`validation FAILED: name: message`). Operators get two nearly-duplicate FAIL lines with different formats in one runlog, which slightly muddies the ?loud single verdict? goal.

- Suggestion: Either drop/replace the older failures print now that the summary line exists, or make the new line the sole PASS path log and keep the detailed `name: message` form only on FAIL (not both).
- Status: open

Issue 4 -- Severity: suggestion

- File: backend/worker/__main__.py:330
 - Description: Stage timings cover checksum / validation / segmentation but not the storage pull (`storage.fetch` of the input video), which is often a large fraction of cloud wall-clock for multi-GB clips. The same mid-run `\_fail()` path therefore still omits the stage that may dominate before checksum starts.
 - Suggestion: Time the fetch into something like `timings["fetch\_s"]` (and optionally push) so runlogs explain full end-to-end latency, not only post-download stages.
 - Status: open
- 30?

Issue 5 -- Severity: nit

- File: backend/worker/__main__.py:374
- Description: On `--skip-validation`, `validation\_payload` is `[]` and is still written as `"validation": []` (because the gate is `is not None`). That is backward-compatible and distinct from omitting the key, but a later CP/P3 consumer cannot tell if dispatcher already validated (skipped here) or if worker ran zero validators.
- Suggestion: Optionally add `"validation\_skipped": true` (or omit `validation` when skipped) so future control-plane ingest of worker verdicts does not treat an empty list as an authoritative empty suite. Not blocking for this observability-only PR.
- Status: open

22. tool_result

```
exit: 0
right_side pairs: 611
backend/pipeline/detectors/native_wasb_runner.py: 64 lines, range 39-426
backend/pipeline/detectors/wasb_infer.py: 35 lines, range 53-1275
backend/worker/__main__.py: 201 lines, range 97-577
tests/test_native_wasb_runner.py: 121 lines, range 672-807
tests/test_worker.py: 190 lines, range 7-378
wrote right-side set
```

23. tool_result

```
exit: 0
bug          backend/pipeline/detectors/native_wasb_runner.py:386 -> INLINE
suggestion   backend/worker/__main__.py:359 -> INLINE
suggestion   backend/worker/__main__.py:369 -> INLINE
suggestion   backend/worker/__main__.py:330 -> INLINE
nit          backend/worker/__main__.py:374 -> INLINE
wrote /tmp/grok-review-pending-8ad8ff23.json (4864 bytes)
inline=5 promoted=0
exit=0
--- stderr ---
--- stdout (truncated) ---
{'id': 4660782651, 'state': 'PENDING', 'html_url': 'https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#pullrequestreview-4660782651', 'user': {'login': 'avidullu', 'id': 1674877, 'node_id': 'MDQ6VXNlcjE2NzQ4Nzc=', 'avatar_url': 'https://avatars.githubusercontent.com/u/1674877?u=12307a198889fe8e2b7def446b332134df436347&v=4', 'gravatar_id': '', 'url': 'https://api.github.com/users/avidullu', 'html_url': 'https://github.com/avidullu', 'followers_url': 'https://api.github.com/users/avidullu/followers', 'following_url': 'https://api.github.com/users/avidullu/following{/other_user}', 'gists_url': 'https://api.github.com/users/avidullu/gists{/gist_id}', 'starred_url': 'https://api.github.com/users/avidullu/starred{/owner}/{repo}', 'subscriptions_url': 'https://api.github.com/users/avidullu/subscriptions', 'organizations_url': 'https://api.github.com/users/avidullu/orgs', 'repos_url': 'https://api.github.com/users/avidullu/repos', 'events_url': 'https://api.github.com/users/avidullu/events{/privacy}', 'received_events_url': 'https://api.github.com/users/avidullu/received_events', 'type': 'User', 'user_view_type':
```

```
'public', 'site_admin': False}, 'commit_id': 'b3dff78311e36ea43c4e0061a17b199396eec8b9'}
id 4660782651
state PENDING
html_url https://github.com/Khelsutra/badminton-highlight-
indexer/pull/550#pullrequestreview-4660782651
```

24. tool_result

```
exit: 0
cleanup done
```

25. assistant

PR #550 review complete

```
**Target:** [PR #550: fix(worker): make the cloud-serving L4 infer path
observable](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550)
**Files reviewed (5):**
- `backend/pipeline/detectors/native_wasb_runner.py`
- `backend/pipeline/detectors/wasb_infer.py`
- `backend/worker/__main__.py`
- `tests/test_native_wasb_runner.py`
- `tests/test_worker.py`

**Diff:** +450 / ?46
```

Overall verdict

Approve with notes. The three L4 observability fixes look correct and well-tested:

```
Gap	Assessment
Validation PASS logging + `report.json`	Additive, backward-compatible
(`timings`/`validation` omitted when absent)	
Per-stage timings	Sensible; mid-run `_fail()` still gets partial timings
Heartbeat (`wasb_infer` ? stdout + stderr merge)	Right root-cause fix for silent INFO + pipe
deadlock |
```

Issues by severity

1 bug · 3 suggestions · 1 nit

- [bug]** `native\_wasb\_runner.py` ? Live-progress `Popen` path never applies `timeout\_sec` (production always passes `progress`, so the 30?min kill is dead code on the streaming path).
- [suggestion]** `worker/\_\_main\_\_.py` ? `n\_checks` reaches into private `\_validators`.
- [suggestion]** Duplicate validation FAIL log lines on failure.
- [suggestion]** Fetch/pull not included in stage timings.
- [nit]** `--skip-validation` writes `"validation": []` with no skipped flag.

PENDING review posted

A **PENDING** review has been created on PR #550 (review id `4660782651`).

- Inline comments:** 5
- Body findings (outside the diff):** 0
- Submit at:** <https://github.com/Khelsutra/badminton-highlight-indexer/pull/550/files> (scroll to **Finish your review?** ? click **Submit review**).

Until you submit, the comments are visible only to you.

26. user

<user_query>

```
pls submit the comments
</user_query>
```

27. assistant

Submitting the pending review on PR #550.

28. tool_result

Failed to call github__pull_request_review_write: failed to get latest review for current user:
Could not resolve to a Repository with the name 'Khelsutra/badminton-highlight-indexer'.

29. tool_result

```
exit: 0
{"body":"## Summary\n\nPR #550 correctly closes three real L4 observability gaps:
validation","id":4660782651,"state":"PENDING","submitted_at":null}
---
avidullu
```

30. tool_result

```
exit: 0
{"html_url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#pullrequestreview
-4660782651","id":4660782651,"state":"COMMENTED","submitted_at":"2026-07-09T07:23:07Z"}
```

31. assistant

Submitted. The pending review is now public as **COMMENT** on PR #550.

```
- **State:** COMMENTED
- **Review:** https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#pullrequestreview-4660782651
- **Inline comments:** 5 (1 bug, 3 suggestions, 1 nit)
```

Authors and other reviewers can see the comments on the PR now.

32. user

```
<user_query>
the author replied. can you take another pass?
</user_query>
```

33. assistant

Fetching the author's replies on PR #550 and reviewing them against the current code.

34. tool_result

```
exit: 1
gh: Bad credentials (HTTP 401)
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
KeyError: 'data'
```

35. tool_result

```
exit: 0
github.com
? Logged in to github.com account avidullu (/home/avidullu/.config/gh/hosts.yml)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
```

```

- Token scopes: 'gist', 'read:org', 'repo', 'workflow'
---
comments exit=0
issue comments exit=0
reviews exit=0
view exit=0
-rw-r--r-- 1 avidullu avidullu 22130 Jul  9 07:42 /tmp/pr550-comments.json
-rw-r--r-- 1 avidullu avidullu  4023 Jul  9 07:42 /tmp/pr550-issue-comments.json
-rw-r--r-- 1 avidullu avidullu  1814 Jul  9 07:42 /tmp/pr550-reviews.json
-rw-r--r-- 1 avidullu avidullu   112 Jul  9 07:42 /tmp/pr550-threads.json
-rw-r--r-- 1 avidullu avidullu 16127 Jul  9 07:42 /tmp/pr550-view.json

```

36. tool_result

```

exit: 0
=== PR ===
fix(worker): make the cloud-serving L4 infer path observable OPEN
e54e4a13e958c7272537855a68e64a028439959c
commits: 2
- b3dff783 fix(worker): make the cloud-serving L4 infer path observable
- e54e4a13 fix(worker): address PR review ? streaming timeout + observability po?

=== Reviews ===
- id=4660782651 avidullu state=COMMENTED @ 2026-07-09T07:23:07Z
  Review submitted: 1 bug, 3 suggestions, 1 nit. Overall approve-with-notes on the three L4
  observability fixes; the main residual risk is the streaming Popen path not enforcing
  timeout_sec.

```

```

=== Issue comments ===

```

```

[avidullu] @ 2026-07-09T07:41:13Z id=4922778714
Thanks for the review ? all five addressed in `e54e4a1`.

```

```

**[bug] streaming path ignored `timeout_sec` (`native_wasb_runner.py`).** Fixed. The live-
progress branch now arms a watchdog `threading.Timer(timeout_sec, ?)` that `proc.kill()`s the
child at the deadline; the kill closes stdout ? the `for line in proc.stdout` read hits EOF ? I
`raise subprocess.TimeoutExpired` so it flows through the same `except` handler as `_run`
(`return None`, `"timed out after Ns"`). So that handler is no longer dead code for the
production native-L4 path, and a silent/wedged child is killed instead of hanging. New test
`test_run_predict_streaming_enforces_timeout` drives a fake child whose stdout read blocks until
the watchdog kills it and asserts `out is None` + the kill fired + the timeout was logged.

```

```

**[suggestion] private `_validators` access (`__main__.py:359`).** Fixed. Added a public
`ValidationRegistry.__len__` and switched to `len(validator_registry)`. (Confirmed nothing uses
the registry in a boolean context, so `__len__` introduces no truthiness surprise.) Test:
`test_registry_len_counts_registered_validators`.

```

```

**[suggestion] two near-duplicate FAIL lines (`__main__.py:369`).** Fixed. The verdict line
stays `[worker] validation FAILED in Ys ? name=FAIL(score)`; the pre-existing detail print is
reworded to `[worker] validation rejected by: name: message` so it complements rather than
repeats. The FAIL test now asserts `count("[worker] validation FAILED") == 1` plus the distinct
`rejected by:` line.

```

```

**[suggestion] pull stage untimed (`__main__.py:330`).** Fixed. `storage.fetch` is now timed
into `timings["fetch_s"]` (and logged on the pull line), so a mid-run `_fail()` explains latency
that accrued before checksum ? often the dominant term for multi-GB clips.

```

```

**[nit] ambiguous `validation: []` on skip (`__main__.py:374`).** Fixed. On `--skip-validation`
the report now carries `validation_skipped: true` and omits the `validation` array, so a CP/P3
consumer can distinguish "dispatcher already validated" from "worker ran a zero-validator
suite". Tests updated for the skip report shape.

```

Local gate re-run green: **1967 passed / 1 skipped**, coverage **84.45%** (floor 70), ruff 0.15.16 + mypy 2.1.0 clean. The live-L4 heartbeat end-to-end still needs an owner CUJ (unchanged caveat).

=== Inline review comments (chronological) ===

[avidullu] path=backend/pipeline/detectors/native_wasb_runner.py line=387 orig=386 id=3549690138 reply_to=None @ 2026-07-09T07:22:34Z

[bug] The live-progress branch (entered whenever `progress` is not None) launches the child with `subprocess.Popen` and then blocks on `for line in proc.stdout` / `proc.wait()` with no timeout. `NativeWasbConfig.timeout\_sec` is only applied on the fast path via `\_run()` ? `subprocess.run(..., timeout=...)`. The worker always passes a progress callback (`trajectory\_hybrid` ? `run\_predict(..., progress=progress)`), so the cloud native-L4 path this PR is about never enforces the configured 30?minute kill. The surrounding `except subprocess.TimeoutExpired` is therefore dead code for production streaming, and the new comment?s ?hangs until timeout? framing is misleading. A child that stops producing output (or wedges after closing pipes) can block the worker indefinitely.

Suggestion: Enforce `timeout\_sec` on the streaming path as well ? e.g. track `deadline = time.monotonic() + timeout`, use a reader thread/`select` with a short poll, or `proc.wait(timeout=remaining)` in a loop and `kill()` + raise/return on expiry (mirroring `\_run`). Add a unit test that a fake Popen which never closes stdout is killed when `timeout\_sec` is small.

[avidullu] path=backend/worker/__main__.py line=None orig=359 id=3549690146 reply_to=None @ 2026-07-09T07:22:34Z

[suggestion] `n\_checks = len(getattr(validator\_registry, "\_validators", []))` reaches into a private registry attribute. That works today, but it is not part of the public validator-registry API and will silently report `0 checks` if the internal field is renamed or if validators are stored differently.

Suggestion: Prefer a public helper (e.g. `len(validator\_registry)` / `list\_registered()` if one exists or is added next to `register`), or log without a count (`[worker] validating?`) to avoid coupling to private state.

[avidullu] path=backend/worker/__main__.py line=384 orig=369 id=3549690148 reply_to=None @ 2026-07-09T07:22:34Z

[suggestion] On validation failure the new summary line (`validation FAILED in ? ? name=FAIL(score)`) is always followed by the pre-existing failures print later in `cmd\_infer` (`validation FAILED: name: message`). Operators get two nearly-duplicate FAIL lines with different formats in one runlog, which slightly muddies the ?loud single verdict? goal.

Suggestion: Either drop/replace the older failures print now that the summary line exists, or make the new line the sole PASS path log and keep the detailed `name: message` form only on FAIL (not both).

[avidullu] path=backend/worker/__main__.py line=332 orig=330 id=3549690152 reply_to=None @ 2026-07-09T07:22:34Z

[suggestion] Stage timings cover checksum / validation / segmentation but not the storage pull (`storage.fetch` of the input video), which is often a large fraction of cloud wall-clock for multi?GB clips. The same mid-run `\_fail()` path therefore still omits the stage that may dominate before checksum starts.

Suggestion: Time the fetch into something like `timings["fetch\_s"]` (and optionally push) so runlogs explain full end-to-end latency, not only post-download stages.

[avidullu] path=backend/worker/__main__.py line=392 orig=374 id=3549690154 reply_to=None @ 2026-07-09T07:22:34Z

[nit] On `--skip-validation`, `validation\_payload` is `[]` and is still written as `"validation": []` (because the gate is `is not None`). That is backward-compatible and distinct from omitting the key, but a later CP/P3 consumer cannot tell ?dispatcher already validated (skipped here)? from ?worker ran zero validators.?

Suggestion: Optionally add `"validation\_skipped": true` (or omit `validation` when skipped) so future control-plane ingest of worker verdicts does not treat an empty list as an authoritative empty suite. Not blocking for this observability-only PR.

Author pushed `e54e4a1` addressing all five findings. Pulling the new diff and re-reviewing the fixes.

38. tool_result

1?You are a meticulous code reviewer. Review code and produce structured review notes in a Markdown file at the path given in the prompt.

Process:

1. Read all relevant code thoroughly
2. Write findings to the specified review notes file
3. Use structured format: severity, file:line, description, suggestion, status

Rules:

- 10?- Check correctness first, style second
- Look for edge cases, error handling gaps, race conditions
- Flag unwrap(), unnecessary clone(), or lock usage
- Be specific: cite file:line for every issue
- Do NOT fix the code yourself
- In your final response, state the file path and summarize the verdict

39. tool_result

exit: 0

REVIEW_ID=29739484

```
{
  "additions": 587,
  "baseRefName": "master",
  "changedFiles": 7,
  "deletions": 49,
  "headRefName": "fix/worker-l4-infer-observability",
  "headRefOid": "e54e4a13e958c7272537855a68e64a028439959c",
  "number": 550,
  "title": "fix(worker): make the cloud-serving L4 infer path observable",
  "url": "https://github.com/Khelsutra/badminton-highlight-indexer/pull/550"
}
```

files:

```
backend/pipeline/detectors/native_wasb_runner.py +65 -20
backend/pipeline/detectors/wasb_infer.py +22 -0
backend/pipeline/validators/base.py +5 -0
backend/worker/__main__.py +137 -27
tests/test_native_wasb_runner.py +162 -2
tests/test_validators_base.py +10 -0
tests/test_worker.py +186 -0
```

39885 /tmp/grok-review-diff-29739484.diff

backend/pipeline/detectors/native_wasb_runner.py

backend/pipeline/detectors/wasb_infer.py

backend/pipeline/validators/base.py

backend/worker/__main__.py

tests/test_native_wasb_runner.py

tests/test_validators_base.py

tests/test_worker.py

From https://github.com/Khelsutra/badminton-highlight-indexer

b3dff78..e54e4a1 refs/pull/550/head -> pr-550-review

e54e4a1 fix(worker): address PR review ? streaming timeout + observability polish

b3dff78 fix(worker): make the cloud-serving L4 infer path observable

00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)

2084d56 fix(storage): build GCSFuse fused path with OS-native separators (#548)

commit e54e4a13e958c7272537855a68e64a028439959c

Author: Avi Dullu <avi.dullu@gmail.com>

Date: Thu Jul 9 13:10:42 2026 +0530

fix(worker): address PR review ? streaming timeout + observability polish

Follow-up to the review on PR #550 (1 bug, 3 suggestions, 1 nit):

- [BUG] The live-progress (streaming) path in NativeWasbRunner.run_predict ? which is the PRODUCTION native-L4 path, since the worker always passes a progress callback ? enforced no timeout: `for line in proc.stdout` / wait() could block forever on a wedged child, and timeout_sec only applied to the fast _run() path. Add a watchdog threading.Timer that kills the child at timeout_sec; the kill closes stdout ? the read loop hits EOF ? we raise through the SAME `except subprocess.TimeoutExpired` handler as _run (return None, "timed out after Ns"), so that handler is no longer dead code here.
- Public validator count: `len(validator\_registry)` via a new ValidationRegistry.__len__, instead of reaching into the private `\_validators` list for the "validating (N checks)" log.
- De-duplicate the FAIL logs: the pre-existing detail print is reworded to "[worker] validation rejected by: ?" so it complements (not repeats) the new "[worker] validation FAILED in Ys ? ?" verdict line.
- Time the storage pull too (`timings["fetch\_s"]`): for a multi-GB clip the download can dominate cloud wall-clock, and a mid-run _fail() now explains latency that accrued before checksum.
- Skip-path report clarity (nit): write `validation\_skipped: true` instead of an ambiguous `validation: []`, so a CP/P3 consumer can distinguish "the dispatcher already validated" from "the worker ran a zero-validator suite".

Tests: streaming-timeout kill via a blocking fake child; ValidationRegistry __len__; fetch_s in the timings set; single-verdict + "rejected by" FAIL lines; validation_skipped on the skip path. Local gate: 1967 passed / 1 skipped, coverage 84.45% (floor 70), ruff 0.15.16 + mypy 2.1.0 clean.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

```
backend/pipeline/detectors/native_wasb_runner.py | 28 ++++++++
backend/pipeline/validators/base.py             |  5 +++
backend/worker/__main__.py                       | 53 ++++++-----
tests/test_native_wasb_runner.py                | 54 ++++++
tests/test_validators_base.py                   | 10 +++++
tests/test_worker.py                            | 22 ++++++--
6 files changed, 153 insertions(+), 19 deletions(-)
fatal: unrecognized argument: --no-stat
{"base":"00e8f92f102283660c25492c103f7665e98810a4","head":"e54e4a13e958c7272537855a68e64a028439959c"}
```

40. tool_result

Subagent started in background.
subagent_id: 019f45d4-8a48-7ff1-9a90-5310c24e89d9
type: general-purpose
description: [reviewer] pr #550 pass2

Use get_command_or_subagent_output with task_ids=["019f45d4-8a48-7ff1-9a90-5310c24e89d9"] and timeout_ms to wait for results.

41. tool_result

```
=== Task 019f45d4-8a48-7ff1-9a90-5310c24e89d9 ===
Command: [subagent:general-purpose] [reviewer] pr #550 pass2
Status: completed
Started: 2026-07-09T07:43:24Z
Ended: 2026-07-09T07:46:52Z
Duration: 208.03s
Exit Code: 0
Output File:
```

=== Output ===

Review notes written to `**~/tmp/grok-review-29739484.md**`.

Verdict

Follow-up is solid. **All five prior issues are RESOLVED** with real code + tests.

```
Prior item	Disposition
Streaming timeout ignored	RESOLVED ? Timer watchdog + `TimeoutExpired` + test
Private `_validators`	RESOLVED ? public `__len__`
Duplicate FAIL logs	RESOLVED ? `"validation rejected by:"`
Untimed fetch	RESOLVED ? `timings["fetch_s"]`
`validation: []` on skip	RESOLVED ? `validation_skipped: true`, omit array
```

New issues

One suggestion (not a ship-blocker): timeout watchdog can false-positive if the timer fires in the tiny window after a successful `wait()` and before `cancel()` ? `timed_out` is set unconditionally, so a clean `rc=0` can still become `TimeoutExpired` / `return None`. Low risk at the default 1800s timeout; worth tightening by only claiming timeout when the child was still alive and/or `rc ? 0`.

Timeout design check (cancel, kill?EOF?raise, daemon timer, no leak on early exit) otherwise looks correct; tests cover merge/line-buffer, tail-on-failure, logging-to-stdout, and kill-on-timeout.

```
<subagent_meta>id=019f45d4-8a48-7ff1-9a90-5310c24e89d9, type=general-purpose, tool_calls=30,
turns=1, duration_ms=208033</subagent_meta>
```

```
<subagent_result>
subagent_id: 019f45d4-8a48-7ff1-9a90-5310c24e89d9
subagent_type: general-purpose
To continue this subagent's conversation, use
resume_from="019f45d4-8a48-7ff1-9a90-5310c24e89d9".
</subagent_result>
```

42. assistant

Second pass complete ? processing findings and posting the review.

43. tool_result

1?## Summary

Second-pass review of PR #550 (head `e54e4a1`) after the author addressed the first review. All five prior items are **RESOLVED** with real implementations and tests, not cosmetic stubs: streaming `timeout_sec` is enforced via a `threading.Timer` watchdog that `proc.kill()`s and raises `TimeoutExpired` into the existing handler; `ValidationRegistry.__len__` is public; FAIL logs are de-duplicated; `fetch_s` is timed; skip path uses `validation_skipped: true` and omits `validation`. Residual risk is a narrow race where a late timer fire after a successful child exit can still set `timed_out` and mis-report success as timeout ? low practical probability at the default 1800s budget, but real correctness gap. No other new regressions found in report.json shape, stderr-merge, logging config, or tests.

Prior issues disposition

- [bug]** streaming Popen path ignored `timeout_sec` ? **RESOLVED**: watchdog `Timer` + `timed_out` Event + `TimeoutExpired` raise; covered by `test_run_predict_streaming_enforces_timeout`.
- [suggestion]** private `_validators` ? **RESOLVED**: `ValidationRegistry.__len__` + `test_registry_len_counts_registered_validators`; worker uses `len(validator_registry)`.
- [suggestion]** duplicate FAIL logs ? **RESOLVED**: second line is `"validation rejected by:"`; test asserts a single `"validation FAILED"` and the distinct prefix.
- [suggestion]** untimed fetch ? **RESOLVED**: `timings["fetch_s"]` + pull log includes duration; round-trip test expects `fetch_s` in report.
- [nit]** `validation: []` on skip ? **RESOLVED**: skip path sets `validation_skipped=true`,

leaves `validation` out of the payload; asserted in `test\_cmd\_infer\_skip\_validation?`.

Issues

Issue 1 -- Severity: suggestion

- File: backend/pipeline/detectors/native_wasb_runner.py:409
- Description: The watchdog callback always does `timed\_out.set()` before `proc.kill()`, and the post-loop check is an unconditional `if timed\_out.is\_set(): raise TimeoutExpired`. If the child exits successfully (rc=0, CSV written) in the narrow window after `proc.wait()` / before `watchdog.cancel()`, a late timer fire can still set `timed\_out` (while `Popen.kill()` is a no-op once `returncode` is set). The run is then reported as a timeout (`return None`) despite a clean completion ? success loses the race to the timer. Cancel-on-completion and daemon Timer are otherwise correct; kill?EOF?raise path works for a truly wedged child; no timer leak when the process dies first.
- Suggestion: Only claim timeout when the child was still alive (and/or when completion was not clean), e.g. set the flag only if `proc.poll()` is None` before kill, and/or after the try/finally treat success as winning the race: `if timed_out.is_set() and not (res is not None and res.returncode == 0): raise TimeoutExpired(...)` . Optionally log the merged `tail` on the timeout path (today `res.stderr` is built then discarded by the raise) for post-mortem context.
- Status: open

20?

44. tool_result

```
exit: 0
backend/pipeline/detectors/native_wasb_runner.py:409 -> INLINE
inline=1 promoted=0
payload written
exit=0
id 4660972357
state PENDING
html_url https://github.com/Khelsutra/badminton-highlight-
indexer/pull/550#pullrequestreview-4660972357
```

45. tool_result

```
exit: 0
done
```

46. assistant

Second pass on PR #550

Reviewed head `**e54e4a1**` (follow-up addressing the first review) against the author's reply.

Prior issues ? all resolved

| # | Original finding | Disposition |
|---|--------------------------------------|--|
| 1 | Streaming path ignored `timeout_sec` | **RESOLVED** ? `threading.Timer` watchdog + `TimeoutExpired` + test |
| 2 | Private `_validators` | **RESOLVED** ? public `ValidationRegistry.__len__` |
| 3 | Duplicate FAIL log lines | **RESOLVED** ? second line is `validation rejected by:` |
| 4 | Untimed storage pull | **RESOLVED** ? `timings["fetch_s"]` |
| 5 | Ambiguous `validation: []` on skip | **RESOLVED** ? `validation_skipped: true`, omit array |

Fixes look real (code + tests), not cosmetic.

New finding (1 suggestion)

****Narrow race on the timeout watchdog**** (`native\_wasb\_runner.py`): the timer always `timed\_out.set()` then `kill()`, and success is reported as timeout if the flag is set after a

clean `rc=0` exit in the window before `cancel()`. Low practical risk at the default 1800s budget, but success can lose a late timer race.

Suggested tightening: only set the flag / raise if the child was still alive (or if completion was not clean).

No other new bugs found in report shape, stderr merge, logging config, or tests.

PENDING review posted

```
- **Inline comments:** 1
- **Body findings:** 0
- **Review id:** `4660972357`
- **Submit at:** https://github.com/Khelsutra/badminton-highlight-indexer/pull/550/files
```

Until you submit, it is visible only to you. Say if you want me to submit it (e.g. as **COMMENT** or **APPROVE**).

47. user

```
<user_query>
pls submit this
</user_query>
```

48. tool_result

```
exit: 0
{"id":4660972357,"state":"PENDING"}
{"html_url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#pullrequestreview-4660972357","id":4660972357,"state":"COMMENTED","submitted_at":"2026-07-09T07:47:45Z"}
```

49. assistant

Submitted. The second-pass review is now public as **COMMENT** on PR #550.

```
- **State:** COMMENTED
- **Review:** https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#pullrequestreview-4660972357
- **Contents:** prior 5 items marked resolved + 1 residual suggestion (timeout watchdog race)
```